Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3799990

PDF Download
3799990.pdf
31 March 2026
Total Citations: 0
Total Downloads: 86

Published: 07 March 2026
Accepted: 13 February 2026
Revised: 29 December 2025
Received: 20 September 2025

Citation in BibTeX format

RESEARCH-ARTICLE

InferCG: Enhancing Python Call Graph Generation via Static Analysis and Large Language Models

DONGMING XIANG, Zhejiang Sci-Tech University, Hangzhou, Zhejiang, China

SHICHAO WU, Zhejiang Sci-Tech University, Hangzhou, Zhejiang, China

HANQIAN TU, Tongji University, Shanghai, China

WANG LIN, Zhejiang Sci-Tech University, Hangzhou, Zhejiang, China

ZUOHUA DING, Zhejiang Sci-Tech University, Hangzhou, Zhejiang, China

Open Access Support provided by:

Zhejiang Sci-Tech University

Tongji University

InferCG: Enhancing Python Call Graph Generation via Static Analysis and Large Language Models

DONGMING XIANG, Zhejiang Sci-Tech University, China
SHICHAO WU, Zhejiang Sci-Tech University, China
HANQIAN TU, Tongji University, China
WANG LIN, Zhejiang Sci-Tech University, China
ZUOHUA DING, Zhejiang Sci-Tech University, China

Call graph construction serves as a foundational component in Python static analysis and software engineering. However, it faces significant challenges due to the language's unique characteristics. That is, the dynamic typing system, extensive use of third-party libraries, and runtime-dependent behaviors in Python create substantial obstacles for traditional static analysis methods. Although these methods typically achieve high precision, they suffer from low recall, making it difficult to strike a balance between soundness and precision. In this paper, we present *InferCG*, a hybrid framework that addresses these limitations by combining coarse-grained static analysis with large language model (LLM)–based semantic reasoning. InferCG first constructs a high-recall candidate call set using permissive name- and arity-based matching, and then employs powerful LLMs such as DeepSeek and Qwen to semantically filter implausible edges. This two-stage design balances recall and precision without requiring dynamic execution or extensive type annotations. Experiments on curated open-source Python projects and the DyPyBench benchmark demonstrate that InferCG outperforms state-of-the-art static tools, which achieves up to 13.9% improvement in average recall and a 5.0% gain in average F1 score over PyCG. Moreover, in a real-world vulnerable-function detection scenario, InferCG enables more accurate reachability analysis than dependency-based tools.

CCS Concepts: • **Software and its engineering** → *Software creation and management*.

Additional Key Words and Phrases: Call Graph, Python, static analysis, LLM

1 INTRODUCTION

Python has emerged as the dominant programming language across key domains, including machine learning, web development, and scientific computing. Its popularity stems not only from a rich ecosystem of libraries but also from its highly expressive syntax and flexible metaprogramming capabilities—such as decorators, dynamic imports, and metaclasses. Although these features significantly enhance developer productivity and program modularity, they simultaneously pose intrinsic challenges for static program analysis, especially in constructing precise and comprehensive call graphs [53, 65].

Call graphs play a foundational role in representing the control-flow and interprocedural data dependencies of a program [32, 53]. As such, they are instrumental in a wide range of downstream software engineering tasks, including code auditing [7], architectural refactoring [32], vulnerability detection [42], and program comprehension. However, it remains an open problem to construct accurate call graphs for Python programs.

Authors' Contact Information: Dongming Xiang, Zhejiang Sci-Tech University, Hangzhou, China, dmxiang@zstu.edu.cn; Shichao Wu, 1556444893@qq.com, Zhejiang Sci-Tech University, Hangzhou, China; Hanqian TU, Tongji University, shanghai, China; Wang Lin, Zhejiang Sci-Tech University, Hangzhou, China; Zuohua Ding, Zhejiang Sci-Tech University, Hangzhou, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2026/3-ART
https://doi.org/10.1145/3799990

Just Accepted

Existing static analysis tools (e.g., PyCG [53] and Pyan [2]) exhibit limited recall when applied to real-world codebases. In particular, these tools consistently underperform in two prevalent scenarios. First, they struggle with cross-module and third-party API invocations, where critical type information and contextual bindings are either absent or only implicitly defined, making it difficult to resolve the correct call targets. Second, they often fail to handle calls to Python's built-in functions, where the lack of explicit type annotations and the polymorphic nature of these functions prevent precise resolution. These dual technical obstacles collectively explain the significant edge omissions prevalent in contemporary call graph construction methodologies. Empirical evidence from Cao et al.'s landmark study [11] corroborates this observation, demonstrating that state-of-the-art Python call graph generators attain merely 68% average recall rates across mainstream open-source projects — a statistically significant coverage deficit with practical ramifications. Crucially, these limitations transcend theoretical concerns: they directly contribute to security audit false negatives, architectural dependency chain fractures, and impaired reasoning capabilities in modern IDE tooling and refactoring ecosystems.

Recent advances in LLMs offer new possibilities for enhancing call graph construction. LLMs such as CodeBERT [21], DeepSeek [24], and GPT-style models [10] have demonstrated impressive capabilities in tasks involving code generation, completion, type inference, and static reasoning [27, 43, 48]. These models are pretrained on massive code corpora, and capture rich semantic and structural knowledge about programming languages. Emerging work has shown that LLMs can implicitly perform type inference and resolve context-dependent expressions [59]. This suggests that LLMs possibly serve as a promising complement to traditional static analysis for recovering hard-to-resolve call edges. For instance, when analyzing an expression like `obj.method()`, an LLM can leverage surrounding code to infer the possible type of `obj` and the corresponding implementation of `method`. This capacity to reason over context and semantics enables LLMs to recognize dynamic or polymorphic call targets that static tools—relying solely on syntactic cues and partial type environments—often miss. As a result, integrating LLM-based inference into the call graph construction pipeline holds the potential to significantly enhance both recall and precision.

In this paper, we present *InferCG*, a Python call graph generation framework that balances high precision and high recall. Our approach combines static analysis with LLMs to address the blind spots of traditional analysis, enabling accurate and comprehensive inference of dynamic call relationships. The framework consists of three stages. First, in the *Function Library Construction* stage, we statically analyze all function definitions, import statements to collect potentially invoked functions, and assemble them into a global function library. Second, during *Coarse-grained Static Analysis*, we use the abstract syntax tree (AST) and the global function library to enumerate all candidate call-pairs, where a call edge may exist. These form a high-recall superset of potential call relationships. Finally, in the *LLM Pruning* stage, we apply an LLM [24, 29] to semantically evaluate each candidate pair. This eliminates false positives through implicit type reasoning and contextual understanding, and retains only the true dynamic call edges.

In summary, our contributions are as follows:

- We introduce InferCG, the first paradigm that integrates LLMs with static analysis for Python call-graph generation. By exploiting rich contextual and type cues in code, InferCG overcomes the precision and completeness limitations of traditional static methods under dynamic features.
- We conduct comprehensive evaluations on both macro benchmarks [28, 53] and dynamic benchmarks [8]. InferCG achieves a superior balance of precision and recall compared to state-of-the-art analyzers [53], and we demonstrate its practical value in vulnerability-function detection for dependency management and security analysis.
- We explore strategies for integrating InferCG with PyCG, investigating trade-offs between analysis accuracy and performance. This study provides a foundation for future hybrid analysis system designs.

Just Accepted

The rest of the paper is organized as follows. Section 2 introduces the background of call graph construction and presents motivating examples. Section 3 describes the implementation details of our proposed approach. Section 4 outlines the research questions and presents the corresponding answers. Section 5 discusses the strengths and limitations of InferCG, followed by threats to validity in Section 6. Section 7 reviews the related work. Finally, Section 8 concludes the paper with a summary of key contributions.

2 BACKGROUND AND MOTIVATION

2.1 Call-Graph Generation Paradigms

Call graphs represent the function-level invocation relationships within a program, and play a foundational role in diverse software engineering tasks, including code understanding, architectural refactoring, and vulnerability detection [3]. The generation of call graphs typically employs two principal methodologies: static analysis techniques that infer calling relationships through source code examination without program execution, and dynamic analysis approaches that capture actual invocation patterns during runtime monitoring.

2.1.1 Static Analysis. Static call-graph generation aims to approximate possible execution paths by inspecting the program's structure. Tools such as `Depends` [1] provide a general-purpose syntactic dependency extraction framework that supports multiple programming languages. It is widely adopted for software visualization, modularization, and architecture analysis. However, `Depends` lacks deep semantic reasoning and performs poorly on language features that require type resolution or inter-procedural context. Within the Python ecosystem, `Pyan` [2] performs AST-based extraction by identifying symbol definitions (e.g., function and method declarations) and their subsequent usages in `Load` contexts. It then applies lexical scoping rules to trace identifiers back to their defining bindings. Although it is effective for small scripts and simple control flow, `Pyan` struggles with inter-procedural call resolution, closures, and dynamic constructs. In contrast, `PyCG` [53] introduces a more sophisticated, context-insensitive static analysis tailored for Python. It builds an intermediate assignment graph that encodes identifier-to-identifier relationships and propagates potential targets across scopes. This design allows `PyCG` to handle advanced features such as higher-order functions, generators, nested functions, multiple inheritance, and complex import hierarchies. Empirical evaluations suggest that `PyCG` achieves better precision and recall than both `Pyan` and `Depends`, particularly in large, modular Python projects.

2.1.2 Dynamic Tracing. In contrast to static analysis, dynamic tracing instruments the program to record concrete function calls during execution. By observing actual runtime behavior, dynamic approaches can precisely capture call edges—including those triggered via reflection, closures, or dynamic dispatch—thereby reducing false positives [5, 47]. However, this precision comes at the cost of coverage: dynamic methods are inherently limited by the input space exercised during test execution [26]. Unexecuted branches, error paths, or infrequently used features may go unobserved, leading to an incomplete call graph. Furthermore, dynamic instrumentation incurs non-trivial runtime overhead and requires a testable environment with representative inputs, which may be unavailable in security-sensitive or legacy systems [65].

2.2 LLMs for Code Understanding and Call-Graph Construction

The recent surge of LLMs has transformed the landscape of program analysis. Pretrained on extensive code corpora, models such as Codex [12], CodeBERT [21], DeepSeek [24], and WizardCoder [40] demonstrate strong capabilities in tasks like type inference, variable resolution, and control-flow reasoning. These models learn both surface syntax and deep semantic regularities, enabling them to infer function behaviors, argument types, and inter-file relationships in ways that surpass traditional static tools.

For call-graph generation, LLMs offer two principal advantages. First, their transformer-based architecture allows context-aware reasoning, making them effective at resolving ambiguous or untyped variables— a pervasive

Just Accepted

challenge in dynamically typed languages like Python. Second, their exposure to a wide range of real-world libraries enables LLMs to internalize common API usage patterns, allowing them to infer likely call targets even for third-party or closed-source modules. For example, an LLM can predict that the call `obj.fit()` likely invokes a scikit-learn model method based solely on variable naming and argument structure.

Nevertheless, it remains an open challenge to leverage LLMs for structured call-graph construction [16, 17, 35–37, 41, 61]. Venkatesh et al. [59] demonstrate that even state-of-the-art models struggle with inter-file reasoning, especially when dealing with indirect calls, dynamic imports, or metaprogramming. Although LLMs can often identify likely callees, their answers in a question-answering paradigm (e.g., "Which functions does `main` call?") may lack completeness or contain hallucinated edges. Additionally, the structured outputs generated via JSON or AST formats often suffer from inconsistencies, including missing fields, syntax errors, or unexpected nesting. These artifacts not only degrade parsing robustness but also compromise downstream tasks that rely on well-formed call graphs.

In light of these limitations, recent efforts have begun exploring hybrid approaches that combine coarse-grained static analysis with LLM-based refinement. By using LLMs to semantically filter or augment candidate edges derived from static techniques, our method aims to achieve better trade-offs between completeness and soundness in real-world codebases.

2.3 Motivating Example

Fig. 1a illustrates a code snippet extracted from the open-source project `schedule` on GitHub. In this example, a class named `Job` defines the method `_schedule_next_run`. Within this method, the expression `datetime.datetime.now()+ self.period` computes a future timestamp by adding an instance `datetime.timedelta` (i.e., `self.period`) to the current time. Here, `datetime.datetime.now()` refers to the method *now*() of the *datetime* class in Python's built-in *datetime* module. The result of this expression, an object `datetime.datetime`, is assigned to `self.next_run`. Subsequently, the method *replace*(...) is invoked on `self.next_run`, taking advantage of the fact that it is an instance `datetime.datetime`. Later in the method, another call to `datetime.datetime.now()` is made, followed by an invocation of the method *time*() on the resulting object `datetime.datetime`.

Although PyCG is a state-of-the-art static analysis tool for Python, its generated call graph (shown in Fig. 1b) of this code snippet exhibits several missing edges when compared to the ground-truth call graph in Fig. 1c. These omissions highlight the limitations of PyCG's analysis capabilities. Specifically, we identify two root causes for the missing call edges:

- **Limited handling of unavailable source modules.** PyCG analyzes only modules for which source code is provided. Calls to functions in closed-source or missing modules are ignored, so edges such as `datetime.datetime.replace` and `datetime.datetime.time` are omitted because PyCG cannot infer the return type of `datetime.datetime.now()`.
- **Lack of built-in type information and semantic modeling.** PyCG's internal representation does not retain sufficient information about built-in types, preventing it from resolving attribute calls that depend on those types (e.g., `list.append()`). Moreover, it does not model the semantics of built-in functions, further undermining call-graph completeness and accuracy.

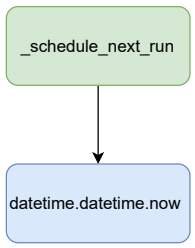
To address these limitations, we propose a hybrid call-graph construction method that integrates static analysis with LLM inference. Traditional static tools like PyCG struggle to infer return types and data-flow propagation across modules in a dynamically typed language, leading to incomplete call graphs. We augment the static analysis with an LLM reasoning engine. That is, by leveraging code-context semantics, function signatures and in-project usage patterns, the LLM generates contextually plausible type hypotheses and infers latent call dependencies. Recent studies (e.g., Venkatesh et al. [59]) have shown that modern LLMs outperform static analyzers on Python

Just Accepted

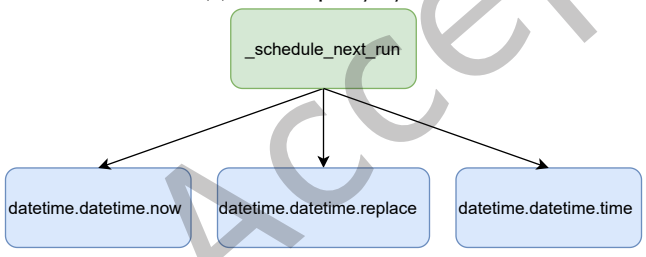
```python
import datetime

class Job(object):
    def __init__(self,  interval: int,  scheduler: Scheduler = None):
        self.period: Optional[datetime.timedelta] = None
    def _schedule_next_run(self):
        self.next_run = datetime.datetime.now() +self.period
        self.next_run = self.next_run.replace(**kwargs)
        now = datetime.datetime.now()
        now.time()
```

(a) Example Code



(b) Call Graph by PyCG



(c) Ground-truth Call Graph

Fig. 1. A Motivating Example

Just Accepted

type-inference tasks, and their contextual awareness enables them to recover implicit type propagation and dynamic calls, offering a promising path to complete, cross-module call graphs.

3 METHODOLOGY

Fig. 2 illustrates the overall architecture of InferCG, which is inspired by Autopruner [34] and Cgpruner [57]. Given a target project, we first resolve its direct dependencies, and then parse all Python source files via the AST to collect every defined function. A coarse-grained static analysis step generates an initial set of caller–callee pairs. To improve precision, InferCG invokes an LLM to filter and prune these candidate edges, removing spurious relationships and yielding a refined call graph.

3.1 Function Library Construction

The first stage of InferCG involves constructing a comprehensive *Function Library (FL)* by statically parsing the source code of a given Python project. This library serves as the foundational knowledge base for subsequent

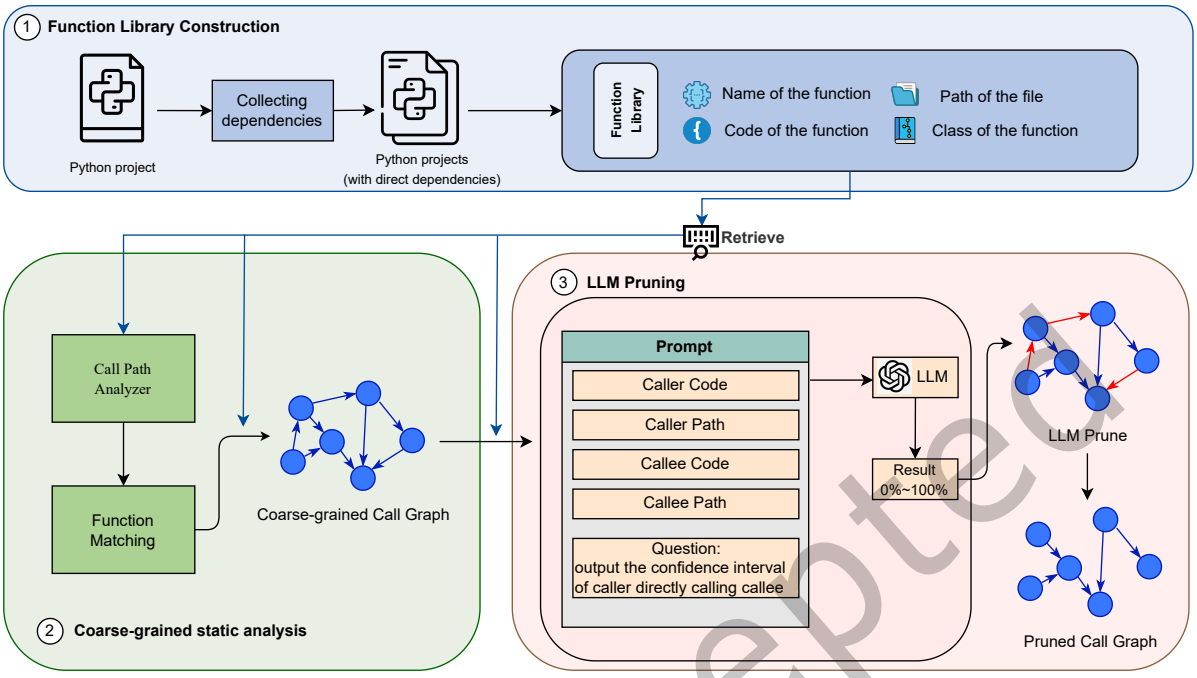


Fig. 2. Overview of InferCG.

Just Accepted

call-graph inference. In contrast to previous approaches [11, 13] that only extract top-level function signatures, our method systematically records three critical elements that enhance semantic reasoning: 1) *nested functions and their lexical scope*, 2) *complete function bodies for contextual analysis*, and 3) *fine-grained import resolution to accurately model inter-module dependencies*.

To populate function library, we traverse all Python source files within the target project directory. Each file is parsed into an abstract syntax tree (AST) using Python's built-in `ast` module. During the traversal, we extract and store the following structural elements:

- **Modules:** For each file, its module name (derived from the project's package structure) and absolute file path are recorded.
- **Classes:** Each class definition are extracted, including its name, base classes (to support inheritance reasoning) and definition location.
- **Functions:** Both top-level and nested functions are collected. For each function, we record its name, the enclosing lexical scope, the host module specification, and full source code body. Nested functions are assigned a fully qualified name (e.g., `A.B.inner`) that reflects lexical scoping.
- **Imports:** All `import` statements and `from...import...` statements are parsed, and their targets are resolved to physical paths via a lightweight static resolution heuristic. This enables accurate modeling of inter-file and inter-package references.

A key enhancement over prior tools lies in the recursive traversal of function bodies to detect and register nested definitions. This ensures local function objects that frequently used as callbacks or closures in real-world Python code are not omitted. By preserving the complete function body as a source string, our method enables downstream LLM modules to perform context-aware semantic inference, such as return type deduction from

nontrivial control flow or interleaved assignments. Additionally, our import resolution component accounts for Python's module import mechanism, including support for relative paths, aliasing, and transitive re-exports. This guarantees that call targets defined in auxiliary files are correctly linked, thereby preventing edge omissions in the inferred call graph. Fig. 3 presents a simplified example from a Python file `test.py`, and our method can generate its function library, as shown in Fig. 4. The output successfully captures all functions, including nested definitions, along with their parameter lists, enclosing classes (if any), file locations, and source code bodies.

```
import numpy as np
import datetime

class ClassA:
    def __init__(self):
        self.m2 = func5
    def m1(self):
        self.m2()

class ClassB:
    def m1(self):
        print("ClassB m1")

def func2():
    class_a = ClassA()
    class_a.func1()

def func3():
    class_b = ClassA()
    class_b.func1()

def func4():
    now = datetime.datetime.now()
    now.time()
    np.array([1, 2, 3])

def func5():
    print("func5")
```

Fig. 3. Code snippet of `test.py`

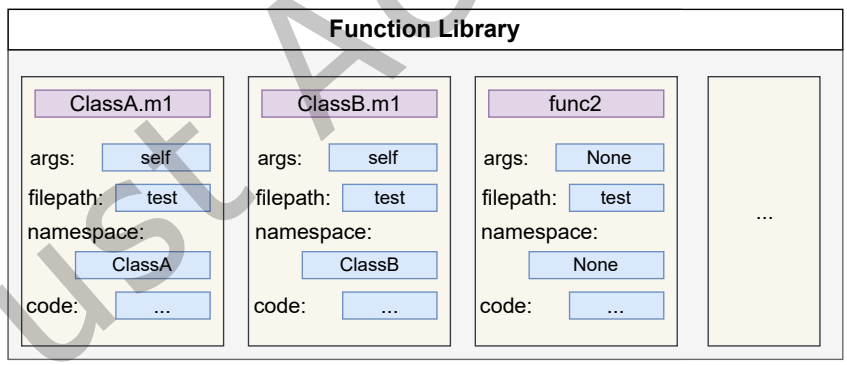


Fig. 4. Example of function-library.

Algorithm 1 is developed to construct the function library. In this algorithm, given a target Python project, we begin by identifying its direct dependencies (*DV*) and merge them with the project itself to form a processing set. We then initialize the function library and the import base. For each element in the processing set, we parse its source code into an AST by using Python's built-in `ast` module. During AST traversal, we process each node according to its type. Specifically, for each `Import` node, we extract and record the import statement in the import base (*IB*). Module-level metadata is collected and stored into *FL*. For every class definition (`ClassDef`), we record

Just Accepted

Algorithm 1 Function Collection

Input: A Python Project p
Output: Function Library FL, Import base IB

```
DV = FL = IB = ∅
DV = getDepends(p) // Get direct dependencies of p
L = {p} ∪ DV
for each l ∈ L do
  AST = parseAst(l) // Parse Python file l into an AST
  for each node ∈ AST do
    if node is an Import node then
      IB = IB ∪ node
    end if
    if node is a Module then
      FL = FL ∪ node
    end if
    if node is a Class Definition then
      FL = FL ∪ node
    end if
    if node is a Function Definition then
      code = getFullFunctionCode(node)
      nestedFuncs = getNestedFunctions(node)
      FL = FL ∪ node
      for each f in nestedFuncs do
        FL = FL ∪ f // Include nested function with scope
      end for each
    end if
  end for each
end for each
return FL
```

the class name, base classes and other structural attributes, and update *FL* accordingly. When encountering a function definition (`FunctionDef`), we add the function to *FL*, extract its complete source body, and recursively examine its content to discover nested functions. These nested functions are registered with their enclosing scopes, and fully qualified names to ensure accurate call context tracking in later stages.

3.2 Coarse-Grained Static Analysis

The second phase of InferCG performs a coarse-grained static analysis to establish an initial over-approximation of the program's call graph. Specifically, for each function in the codebase, we identify a set of potential callee functions that may be invoked. This phase prioritizes *recall* over *precision*. That is, it aims to include as many true positive call edges as possible, at the cost of tolerating false positives. These candidates will be refined in subsequent stages via LLM-based inference.

Traditional static tools such as `PyCG`, `Depends`, and `Pyan` can be used to produce such initial call graphs. However, they often suffer from poor coverage, especially in the presence of dynamic language features. Their limitations include: 1) missing calls to third-party libraries due to incomplete or unavailable source code, and 2) failing to resolve invocations of built-in functions or class methods due to the lack of runtime type inference [1, 2, 53].

Just Accepted

To address these shortcomings, our approach adopts a permissive AST-based analysis that leverages contextual information such as variable assignments and import resolution to extract a superset of candidate callees with improved recall.

Fig. 5 illustrates the overall workflow of the Coarse-Grained Static Analysis module, which is divided into two major components: the *Call-Path Analyzer* and *Function Matching*. The process begins by parsing the project's source code into ASTs. The Call-Path Analyzer then performs three sequential analyses over the AST: *Import Analysis* to resolve module aliases, *Assignment Extraction* to capture variable-to-object bindings, and *Call Resolution* to reconstruct fully qualified call expressions. In the Function Matching phase, each call expression is compared against known function signatures from the Function Library and the Built-in Catalog based on name and arity. The matched edges are aggregated to form the final Coarse-Grained Call Graph.

At the implementation level, the coarse-grained analysis of InferCG exhibits a fundamental divergence from PyCG. As a state-of-the-art static analysis tool, PyCG centers on constructing a fine-grained Assignment Graph. It employs context-insensitive interprocedural analysis to track assignment relationships among identifiers (variables, functions, and classes), thereby resolving call targets. While this approach achieves high precision in ideal scenarios, its effectiveness heavily relies on the complete availability of source code and accurate type inference. In contrast, InferCG adopts a distinct design philosophy and implementation trajectory. The coarse-grained phase of InferCG does not aim for immediate precision; instead, it prioritizes high recall. It intentionally embraces over-approximation, delegating the burden of precision refinement to the subsequent LLM-based pruning stage. Technically, whereas PyCG necessitates the construction of complex intermediate representations and deep data flow propagation, InferCG relies on lightweight AST traversal and heuristic matching. Rather than performing deep propagation, InferCG establishes alias mappings by parsing import statements, extracts assignment relationships to infer potential bindings, and matches targets directly based on function names and arity. Although this method is computationally more efficient, its tolerant matching mechanism enables it to cover highly dynamic behaviors—such as duck typing and runtime function dispatch—that often remain blind spots for PyCG due to its static nature. While this approach inevitably introduces noise, these ambiguous cases serve as the focal point for the subsequent LLM phase, which leverages semantic understanding to filter false positives.Furthermore, InferCG addresses PyCG's inability to resolve calls to compiled extension modules or Python built-in methods through two mechanisms: (a) constructing a Function Library containing direct dependencies to enable approximate matching even when full source code is absent; and (b) maintaining a Built-in Catalog to directly match common built-in method calls by name.

3.2.1 Call-Path Analyzer. The core of this analysis lies in a dedicated *Call-Path Analyzer*, responsible for resolving static call paths *within individual functions* across the project's source code. Built on top of Python's standard `AST` framework, this analyzer deliberately differs from tools such as PyCG, which propagate call-path information across function boundaries to construct whole-program call graphs. While such inter-procedural propagation can improve coverage, it also incurs substantial memory overhead in large or dependency-heavy Python projects, and may lead to scalability issues or even out-of-memory failures in complex settings. In contrast, our analyzer explicitly restricts its scope to intra-procedural analysis: it does not propagate call-path information across functions at this stage. Instead, it focuses on reconstructing lightweight semantic relationships among call expressions that occur within the same function body, trading global completeness for improved robustness and scalability.

The first component performs *import analysis*, identifying and resolving all `import` and `from ... import ...` statements within each source file. Its primary task is to construct an alias map that rewrites shorthand module references into their fully qualified counterparts (e.g., resolving `np` to `numpy`). This alias resolution provides the necessary module-level context for disambiguating call expressions involving imported modules and third-party packages, but does not attempt to infer how imported symbols propagate across function calls.

Just Accepted

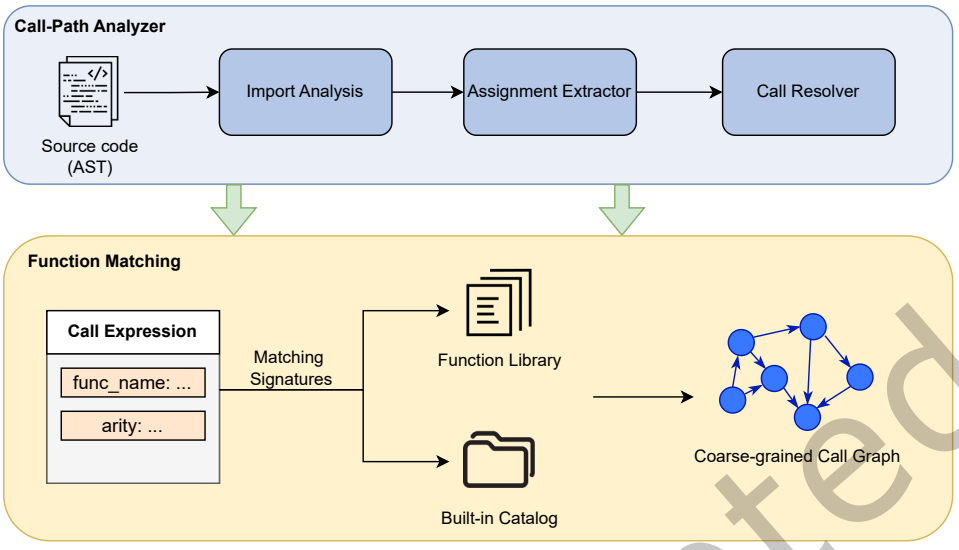


Fig. 5. Coarse-Grained Static Analysis.

Next, the *assignment extractor* processes variable assignments that occur locally within a function body to infer potential type or module bindings. It pays special attention to object instantiations and attribute chains in constructor calls. For example, an assignment like `cfg = ConfigParser()` leads to a local association between `cfg` and `configparser.ConfigParser`. To improve coverage, the extractor recursively traverses nested expressions within the same function (e.g., `argparse.ArgumentParser().add_argument()`), and builds a function-scoped assignment map to aid subsequent call resolution. Importantly, assignments and inferred bindings are not propagated across caller–callee relationships.

Finally, the *call resolver* inspects each function-call expression and reconstructs its invocation path using the alias and assignment maps derived from the enclosing function scope. When analyzing a call such as `cfg.read()`, the analyzer leverages the locally inferred type of `cfg` to resolve the invocation as `configparser.ConfigParser.read`. This resolution process involves backtracking along attribute chains within the same function until a root object or module is identified, without relying on inter-procedural data-flow information. As a result, the resolver provides a conservative over-approximation of potential callees for object-oriented method invocations and library calls.

By combining these three layers of intra-procedural analysis—import resolution, local variable binding extraction, and call-path reconstruction—the Call-Path Analyzer produces a *precise but incomplete* set of call graph candidates. Because the analysis is confined to individual function bodies and does not track value flows or dispatch behavior across function boundaries, the resulting call graph favors precision within its local scope, while inevitably missing call relationships that require inter-procedural reasoning or runtime information. This deliberately restricted design improves scalability and reduces spurious edges at the coarse-grained stage, but it also leads to limited recall at the whole-program level.

3.2.2 Coarse-Grained Function Matching. A coarse-grained function matching mechanism supplements the call graph by heuristically associating function call expressions with likely callee candidates based on syntactic features. While the Call–Path Analyzer is capable of resolving many call paths through explicit static resolution, it remains inherently limited in the face of Python's dynamic semantics. Features such as duck typing, higher-order

Just Accepted

Algorithm 2 Coarse-Grained Static Call Graph Construction

Input: Function Library FL, Built-in Catalog BiC
Output: Call graph $G = (V, E)$

```
V = ∅, E = ∅// Initialize the set of nodes and edges
for each f ∈ FL do
    AST = parseAst(f)
    A = ∅ // Initialize alias map from import statements
    M = ∅ // Initialize assignment map for variable-to-object tracking
    for each node ∈ AST do
        if node is Import then
            A = A ∪ resolveImport(node) // Resolve import aliases
        end if
        if node is Assignment then
            /* Track variable assignments to infer potential object or module bindings */
            M = M ∪ resolveAssignment(node)
        end if
        if node is Call Expression then
            /* Identify the function scope enclosing this call site */
            caller = getEnclosingFunction(node)
            /* Reconstruct the fully qualified call path using alias and assignment maps */
            path = resolveCallPath(node, A, M)
            V = V ∪ {caller, path}
            for each f′ ∈ FL do
                if f′.name = node.name and f′.arity = node.arity then
                    E = E ∪ {(caller, f′)}
                end if
            end for each
            if node.name ∈ BiC then
                E = E ∪ {(caller, BiC(node.name))}
            end if
        end if
    end for each
end for each
return G = (V, E)
```

functions, and dynamic function bindings often elude deterministic resolution from syntax trees alone. As a remedy, we introduce a lightweight fallback strategy that boosts recall by exploiting the structural metadata stored in the Function Library described in Section 3.1. This matching mechanism operates by querying the FL for functions whose *name* and *arity* (i.e., number of arguments) match those of a given call expression. If such a match is found, an edge is conservatively added to the static call graph. This process is particularly effective for resolving ambiguous method calls, such as those on dynamically typed objects or built-in wrapper types. For instance, consider the invocation of `datetime.datetime.replace` in Fig. 1a, where the receiver's type is not statically inferable. Even though the Call–Path Analyzer cannot resolve this call path, our fallback matcher identifies a function named `replace` in the Function Library with a matching arity and links it as a candidate callee. This strategy introduces some imprecision, as it may incorrectly associate unrelated functions with similar signatures. For example, if the

Just Accepted

Function Library contains both `datetime.datetime.replace` and `DataFrame.replace`, and the module where `_schedule_next_run` resides imports both `DataFrame` and `datetime`, then both `datetime.datetime.replace` and `DataFrame.replace` will be regarded as potential callees of `_schedule_next_run`. Nonetheless, it substantially increases the likelihood of covering semantically relevant call edges that would otherwise be omitted. In practice, this results in a richer call graph that enables downstream refinement modules—such as LLM-based filters—to operate with broader semantic context.

By complementing the explicit resolution logic of the Call–Path Analyzer, the coarse-grained function matcher fills important gaps in the static call graph and enhances the robustness of call-graph construction in dynamic programming environments.

3.2.3 Built-in Function Matching. A built-in function matching mechanism incorporates standard library behaviors into the call graph using syntactic heuristics. Python's built-in functions and methods (e.g., `len`, `sorted`, `list.append`, and `dict.get`)constitute an essential part of real-world Python programs. However, these functions are not defined in user code and are thus absent from the Function Library, which focuses solely on user-defined and third-party functions extracted from source code. Consequently, relying solely on Function Library-based resolution would lead to systematic under-representation of built-in invocations. To bridge this gap, we maintain a curated catalog *BiC* of Python built-in functions and methods, each entry annotated with its canonical name and associated type or module (e.g., `list.append` → `builtins.list`). During AST traversal, each call expression is checked against this catalog. If a match is found—based on exact syntactic name correspondence—a candidate edge is emitted from the calling function to the corresponding built-in entry in *BiC*. This matching procedure is purely name-based and intentionally avoids dependence on dynamic type inference, which would require runtime instrumentation.

Including built-in functions in the static graph not only improves recall but also enhances the downstream utility of the graph for tasks such as dependency analysis, semantic modeling, and type inference. In effect, the enriched call graph offers a more holistic representation of program behavior, thereby supporting more accurate and semantically meaningful inference in later stages.

3.2.4 Algorithm for Coarse-Grained Call Graph Construction. Algorithm 2 shows the process of Coarse-Grained Static Analysis. It parses each source file by parsing its AST, tracking aliases from import statements and variable assignments, and matching function calls against known definitions in the Function Library and a catalog of built-in methods (*BiC*). Although this step produces a imprecise call graph, it guarantees a broad coverage of potential call targets—including those unresolved by prior static tools. Let N denote the number of functions in the Function Library (including both project-defined and dependency-provided functions), and let K represent the average number of AST nodes per function. The algorithm visits each function once and traverses its AST, incurring a baseline cost of $O(N \cdot K)$. For each function-call expression, it performs a linear scan over all function definitions in FL to find name- and arity-based matches, resulting in an additional cost of $O(N^2 \cdot C)$, where C is the average number of call expressions per function. In the worst case where $C = O(K)$, the total time complexity becomes $O(N^2 \cdot K)$. The space complexity is $O(N^2)$, dominated by the number of edges in the coarse-grained call graph.

3.3 LLM Pruning

To enhance the precision of the statically constructed call graph, we introduce a refinement stage—termed *LLM-Based Pruning*—that leverages an LLM as a discriminative filter for candidate call edges. The central idea is to prompt the LLM to determine, for each caller–callee pair produced by the coarse-grained analysis (Section 3.2.2), whether a genuine invocation exists. Our coarse-grained static analysis is designed to favor high recall, which naturally leads to conservative over-approximation and the inclusion of many false-positive call edges—especially

Just Accepted

in the presence of dynamic language features such as duck typing, reflection, and polymorphic dispatch. Although this recall-oriented design ensures broad coverage, it can hinder downstream tasks that rely on high-precision call graphs (e.g., program comprehension, slicing, and vulnerability detection). By leveraging the LLM's capacity for contextual understanding and type reasoning, our goal is to refine these coarse over-approximations through selective, semantically informed edge pruning.

Beyond eliminating spurious edges, the LLM-based pruning stage enables InferCG to address several classes of cross-module type resolution challenges that are difficult to handle with purely static techniques. First, the LLM can exploit its prior knowledge of widely used libraries and APIs to infer the concrete types of objects returned by external functions. For instance, when a variable is assigned from a call to a third-party library function whose return type is not explicitly annotated, static analysis alone may fail to determine the object's type. In contrast, the LLM can recognize common library semantics and use this knowledge to correctly associate subsequent method invocations with their defining classes, thereby disambiguating cross-module calls involving external dependencies. Second, the LLM can disambiguate method calls on dynamically typed objects when multiple imported modules define identically named methods with compatible signatures. In such cases, coarse-grained name-and-arity matching yields multiple candidate callees across different modules or classes. By reasoning over the surrounding function body, including variable usage patterns, control-flow context, and semantically meaningful identifiers, the LLM can infer the most plausible object type and select the corresponding method implementation. This capability allows InferCG to resolve certain cross-module ambiguities that arise from Python's flexible typing and import mechanisms, even when static analysis cannot conclusively bind a variable to a single class.

Informed by our empirical observations over diverse Python projects, we found that local code context alone, such as the bodies of the caller and callee functions, is often insufficient for accurate call resolution. Many call relations depend on global or structural information (e.g., import statements, module-level declarations, and initializer logic within classes) that affects name resolution and runtime behavior. Accordingly, the LLM prompts are augmented with carefully selected contextual information, including relevant import statements, class initializers, and module-level definitions. This enriched context enables the LLM to reason about cross-module type relationships while remaining lightweight and scalable. At the same time, InferCG does not aim to fully reconstruct global class hierarchies or perform exhaustive inter-procedural type inference; instead, it relies on the LLM to resolve a subset of practically important cross-module ambiguities that commonly arise in real-world Python code.

3.3.1 Prompt Design for Non-Built-In Functions. For functions defined within the codebase or imported from third-party dependencies, we craft prompts that encapsulate the syntactic and semantic footprint of the candidate caller–callee pair.

The specific prompt format is as follows:

```
## non-builtin prompt

You are given two code snippets and asked to determine whether there is a direct
    invocation from the first (caller) to the second (callee).

Caller name: {caller}
Caller file path: {caller_file_path}
Caller code: {caller_code}

Callee name: {callee}
Callee file path: {callee_file_path}
Callee code: {callee_code}
```

Just Accepted

```

Please analyze whether {caller} invokes {callee} as follows:
1. Examine the structure of the caller code and identify any invocations (function
    calls or object instantiations) that may correspond to the callee.
2. If the callee is a class, consider only those cases where the class is
    explicitly instantiated (e.g., ClassName()).
3. Based on your reasoning and understanding of the provided code, estimate the
    confidence (0% to 100%) that {caller} directly invokes {callee}.
```

3.3.2 Prompt Design for Built-In Functions. Built-in function invocations introduce additional ambiguity due to the polymorphic nature of their method names. Methods such as such as `clear`, `append`, and `get` are overloaded across multiple types (e.g., `list.clear`, `set.clear`, and `dict.get`), yet their behaviors differ fundamentally based on the receiver's type. As described in Section 3.2.3, our static analysis conservatively overmatches such cases using purely name-based heuristics, which frequently leads to false positives.

To address this, we construct a specialized prompt for built-in function calls that incorporates two stages:

(1) **Receiver Type Inference:** We first ask the LLM to infer the type of the object on which the method is invoked, based on surrounding assignment statements, constructor usage, and class attributes.
(2) **Call Plausibility Check:** We then check whether the inferred type semantically supports the built-in method in question. This ensures that the invocation is valid for that object type.

The corresponding prompt structure is illustrated below:

```
## builtin prompt

You must decide if there is a direct invocation between two pieces of code.
Given a caller's name and code, and a callee's name and code, determine whether
    the caller invokes the callee. The callee uses the notation Type.method.

For example:
<PyStr>.join means that the str object calls the join method
<PyList>.append means that the list object calls append
<PyDict>.clear means that the dict object calls the clear method
<PySet>.add means that the set object calls the add method
<PyTuple>.count means that the tuple object calls the count method
<PyNum>.bit_length means that the number object calls the bit_length method
<PyFile>.read means that the file object calls the read method
<builtin>.print means that the Python built-in function print is called

Caller name: {caller}
Caller code: {caller_code}

Callee name: {callee}
Callee code: {callee_code}

Please analyze step by step whether {caller} invokes {callee}:
1. Infer the most likely data type of the object on which {callee} is invoked
    within {caller}.
2. Based on your type inference, estimate the confidence (0% to 100%) that {caller
    } directly calls {callee}.
```

Just Accepted

Although the prompts for built-in and non-built-in functions share a similar high-level structure, we intentionally design them as two distinct templates due to fundamental differences in the sources of ambiguity they must resolve. For non-built-in functions, call disambiguation primarily depends on syntactic scope, import resolution, and the semantic relationship between user-defined or library-defined functions. In such cases, providing the caller and callee code bodies, together with their file-level context, is generally sufficient for the LLM to assess call plausibility. In contrast, built-in function calls exhibit a qualitatively different form of ambiguity that centers on *receiver-type resolution*. Many built-in methods (e.g., `append`, `clear`, `get`) are shared across multiple core data types with distinct semantics, and name-based static matching alone systematically over-approximates these cases. Empirically, we found that using a unified prompt often led the LLM to shortcut the reasoning process by relying on surface-level name similarity, resulting in incorrect confirmations of infeasible built-in calls. To mitigate this effect, the built-in prompt explicitly enforces a two-stage reasoning procedure—first inferring the receiver's data type, and then validating whether the method is semantically supported by that type—which substantially improves disambiguation reliability.

Together, these two prompting strategies for Built-In Functions and Non-Built-In Functions allow InferCG to retain high-recall candidate graphs while improving their precision through principled LLM-based validation, thereby achieving a more accurate and semantically faithful representation of the program's call structure.

4 EVALUATION

4.1 Experimental Settings

Benchmark Selection. To evaluate the effectiveness and generalizability of our approach in constructing Python call graphs, we conduct experiments on two benchmarks that reflect both curated and real-world codebases. Our first benchmark is sourced from two prior studies[53] [28], and it comprises 11 open-source Python projects with high-quality, as manually curated ground-truth call graphs. In this benchmark, the first five projects are taken directly from Salis et al. [53], who used them as the entire dataset for their original evaluation. The selection process employed a rigorous methodology: it first identified popular Python repositories on GitHub (ranked by star count), then applied a size filter to exclude projects exceeding 3,500 lines of code, thereby focusing on medium-scale codebases. This systematic approach yielded five representative repositories. For each selected project, it dedicated approximately 10 person-hours to conduct manual source code analysis and construct precise, function-level call graphs in JSON format. By including all five original projects, we ensure a fair and reproducible comparison with the prior work. The remaining six projects are drawn from Huang et al. [28], collectively forming the complete benchmark suite used in their research. These repositories span a range of application domains, including data processing, web development, and machine learning. Each project was manually annotated with fine-grained caller–callee relationships, and independently cross-validated by multiple annotators. By incorporating the full datasets from both prior studies, this benchmark ensures comprehensive coverage and eliminates the risk of selection bias.

In addition to the original curated benchmark, we also create a corrected version of this dataset, which we refer to as the *corrected macro-benchmark*. During our evaluation, we identified that several call edges marked as false positives were in fact legitimate but missing from the original annotations. To address this gap, we performed a systematic multi-reviewer validation over all candidate call edges produced by the evaluated tools and updated the ground-truth set accordingly.

To complement the curated benchmark and assess scalability and robustness under more realistic conditions, we additionally adopt a second benchmark drawn from DyPyBench [8]. DyPyBench contains over 50 real-world Python projects selected for their diversity in code structure, functionality, and external dependencies. Because PyCG in DyPyBench could only successfully run on 39 projects, we use these 39 projects for our experiments. These projects include widely-used libraries and applications across different domains, e.g., CLI tools, scientific

Just Accepted

computing, and data visualization, which provide a challenging testbed for both static and hybrid approaches. Importantly, we use the same code versions and experimental configurations as specified in DyPyBench to ensure a fair and consistent comparison. This includes matching Python interpreter versions, dependency specifications, and the same preprocessing pipelines where applicable. Overall, the combination of curated and real-world benchmarks provides a comprehensive basis for evaluating the overall performance of our method across diverse Python software projects.

The datasets supporting the findings of this study are available in the InferCG repository at https://github.com/thusloop/InferCG.

LLM Selection. To support high-precision call graph refinement, we select two representative LLMs for our experiments: Qwen-max [29] and DeepSeek-R1 [24]. By leveraging both models, our experiments aim to evaluate how different LLM design philosophies affect call graph pruning and inference quality. These models are used in consistent configurations across all experiments, ensuring a fair comparison of their impact.

Baseline Approaches. We compare our method InferCG with two existing tools PyCG [53] and Depends [1]. PyCG is a state-of-the-art tool for constructing call graphs in Python and recognized for its strong empirical performance. Depends is a static analysis tool designed to extract source code dependencies by inferring syntactic relationships between entities such as files and methods across multiple programming languages. These tools represent the state-of-the-practice for static call graph generation, and serve as strong baselines for evaluating the effectiveness of hybrid methods.

Evaluation Metrics. To quantify the quality of the generated call graphs, we adopt three widely used metrics: precision (P), recall (R), and F1 Score ($F1$). They are computed as follows:

$$\mathrm{P} = \frac{|S \cap G|}{|S|}, \quad \mathrm{R} = \frac{|S \cap G|}{|G|}, \quad \mathrm{F1} = \frac{2 \cdot P \cdot R}{P + R}$$

, where S is the set of predicted function call edges and G denotes the set of ground-truth edges derived from manual annotations. Precision reflects the proportion of correctly predicted edges among all predicted ones, recall measures the proportion of ground-truth edges successfully recovered, and F1 balances both dimensions.

> In this work, we place particular emphasis on the F1 score, as it captures the trade-off between precision and recall. In experiments, due to the fact that InferCG maximizes recall through coarse-grained static candidate generation and improves precision via LLM-based pruning, the F1 score demonstrates the overall effectiveness of our approach.

Research Questions (RQs). Our experimental design is explicitly structured to investigate the following research questions, with each methodological component systematically addressing specific aspects of these inquiries.

- **RQ1:** Is InferCG effective in constructing Python call graphs?
- **RQ2:** Can InferCG assist in identifying vulnerable functions that are actually invoked?
- **RQ3:** What is the contribution of LLM-based pruning and global context information to InferCG's effectiveness?
- **RQ4:** How effectively does the integration of InferCG and PyCG complement each other to improve call graph construction?

4.2 Effectiveness of InferCG (RQ1)

4.2.1 Macro-Benchmark Results. To evaluate the overall effectiveness of InferCG in constructing Python call graphs, we conduct a comprehensive comparison against two representative static analysis tools, PyCG and

Just Accepted

Depends. We use 11 macro-benchmark projects with manually curated ground-truth annotations. To prevent information leakage during hyperparameter tuning, we partition the projects into a training set and a test set. The training set consists of six projects: `fabric`, `face_classification`, `Sublist3r`, `bpytop`, `furl`, and `rich_cli`. The remaining five projects — `asciinema`, `autojump`, `sqlparse`, `sshtunnel`, and `textrank4zh` — constitute the test set. The confidence threshold of InferCG is tuned on the training set, following the practice of prior work (e.g., AutoPruner [34]). As shown in Figure 6, the 75% threshold yields the highest F1 score on the training set. Consequently, we adopt 75% as the default threshold for InferCG. Table 1 reports the performance of InferCG (instantiated with two LLMs, DeepSeek and Qwen) and the baseline tools on the test set under this fixed threshold, demonstrating a favorable balance between precision and recall.

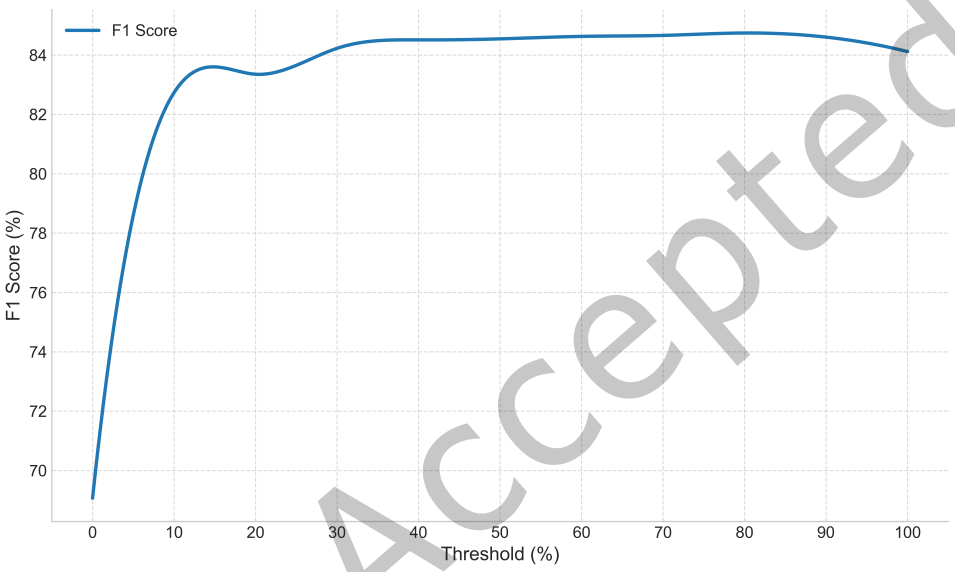


Fig. 6. DeepSeek: F1 at different confidence thresholds

Just Accepted

Across all projects, InferCG achieves consistently higher recall and F1 scores compared to traditional tools, highlighting its advantage in recovering a greater portion of actual call relationships. This advantage is particularly pronounced in scenarios involving ambiguous method resolution across modules. In our macro-benchmark, we observe that approximately 30% of call sites involve methods with identical names defined in multiple classes or modules, which poses a substantial challenge for purely static, name-based resolution strategies. Such ambiguities frequently arise in real-world Python code due to extensive reuse of common method names (e.g., `save`, `load`, `read`, `fit`) across libraries and application-specific abstractions. InferCG addresses a significant fraction of these ambiguous cases during the LLM-based pruning stage by leveraging contextual and semantic cues that are unavailable to conventional static analysis. For example, when a variable is assigned from a call to an external library function whose return type is implicit (e.g., `df = pd.read_csv(...)`), static analysis cannot precisely determine the concrete type of `df`. In contrast, the LLM can exploit its prior knowledge of widely used libraries to infer that `read_csv` returns a `pandas.DataFrame`, and consequently resolve subsequent calls such as `df.head()` to the correct class method. Similarly, in cases where multiple imported modules define methods with the same name and compatible signatures, the LLM can reason over the surrounding function logic, variable usage patterns, and semantically meaningful identifiers to infer the most plausible receiver type and select the appropriate

Table 1. Comparison of call graph construction tools on the macro-benchmark (Precision / Recall / F1, in %).

Set	Project	Depends			PyCG			InferCG (DeepSeek)			InferCG (Qwen)		
		Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
Train	fabric	82.1	6.0	11.2	92.2	61.9	74.1	76.7	77.3	77.0	75.7	72.3	74.0
Train	face_classification	97.0	7.3	13.6	99.5	88.4	93.6	98.3	92.9	95.5	97.1	90.4	93.6
Train	Sublist3r	100.0	21.1	34.8	99.2	58.4	73.5	92.7	86.4	89.5	92.1	84.5	88.1
Train	bpytop	99.5	26.2	41.4	99.4	73.3	84.4	84.5	78.4	81.3	82.4	72.7	77.2
Train	furl	100.0	28.3	44.1	99.5	56.5	72.1	89.0	67.6	76.8	90.5	67.9	77.6
Train	rich_cli	83.3	15.2	25.6	95.8	69.7	80.7	91.8	84.8	88.2	92.5	74.2	82.4
Test	asciinema	100.0	18.2	30.7	97.8	65.4	78.4	89.9	81.8	85.7	89.8	81.6	85.5
Test	autojump	99.2	24.3	39.0	96.6	66.1	78.5	89.9	79.0	84.1	90.6	74.2	81.6
Test	sqlparse	94.8	24.5	39.0	95.2	54.1	69.0	86.9	73.8	79.8	87.7	73.2	79.8
Test	sshtunnel	100.0	36.6	53.6	100.0	70.9	83.0	82.3	86.6	84.4	78.3	83.7	80.9
Test	textrank4zh	100.0	16.1	27.7	100.0	77.0	87.0	88.2	86.2	87.2	88.2	86.2	87.2
Train Avg	–	93.7	17.3	28.5	97.6	68.0	79.7	88.8	81.2	84.7	88.4	77.0	81.8
Test Avg	–	98.8	24.0	38.0	97.9	66.7	79.2	87.4	81.5	84.2	86.9	79.8	83.0
Overall Avg	–	96.0	20.3	32.8	97.8	67.4	79.5	88.2	81.3	84.5	87.7	78.3	82.5

callee. These disambiguation capabilities directly contribute to InferCG's improved recall. On average, InferCG with DeepSeek attains a recall of 81.5%, representing a substantial improvement of 14.8 percentage points over PyCG and 57.5 percentage points over Depends. The results indicate that a non-trivial portion of the recovered call edges correspond to ambiguous call sites. At the same time, we emphasize that not all ambiguous cases can be fully resolved: when static context is insufficient and multiple candidates remain semantically plausible, such ambiguities may persist. Nevertheless, the prevalence of same-named method call sites in the benchmark, together with the observed recall gains, underscores the practical importance of LLM-assisted disambiguation for real-world Python call graph construction.

Table 2. Evaluation on the corrected macro-benchmark, where values in parentheses indicate the changes relative to Table 1.

Set	Project	InferCG (DeepSeek)			InferCG (Qwen)			PyCG		
		Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
Train	fabric	87.3 (+10.6)	79.5 (+2.2)	83.2 (+6.2)	86.1 (+10.4)	74.3 (+2.0)	79.7 (+5.7)	92.2 (–)	55.9 (−6.0)	69.6 (−4.5)
Train	face_classification	98.8 (+0.5)	93.0 (+0.1)	95.8 (+0.3)	97.5 (+0.4)	90.5 (+0.1)	93.9 (+0.3)	99.5 (–)	88.0 (−0.4)	93.4 (−0.2)
Train	Sublist3r	94.5 (+1.8)	86.7 (+0.3)	90.4 (+0.9)	93.9 (+1.8)	84.8 (+0.3)	89.1 (+1.0)	99.2 (–)	57.4 (−1.0)	72.7 (−0.8)
Train	bpytop	91.4 (+6.9)	80.2 (+1.8)	85.4 (+4.1)	87.9 (+5.5)	73.4 (+0.7)	80.0 (+2.8)	99.4 (–)	69.3 (−4.0)	81.7 (−2.7)
Train	furl	92.9 (+3.9)	68.5 (+0.9)	78.9 (+2.1)	93.7 (+3.2)	68.2 (+0.3)	78.9 (+1.3)	99.5 (–)	54.0 (−2.5)	69.6 (−2.5)
Train	rich_cli	98.4 (+6.6)	85.7 (+0.9)	91.6 (+3.4)	98.1 (+5.6)	74.3 (+0.1)	84.6 (+2.2)	95.8 (–)	65.7 (−4.0)	78.0 (−2.7)
Test	asciinema	94.9 (+5.0)	82.6 (+0.8)	88.4 (+2.7)	94.3 (+4.5)	81.8 (+0.2)	87.6 (+2.1)	97.8 (–)	62.5 (−2.9)	76.3 (−2.1)
Test	autojump	92.5 (+2.6)	79.9 (+0.9)	85.8 (+1.7)	92.9 (+2.3)	74.8 (+0.6)	82.9 (+1.3)	97.5 (+0.9)	65.5 (−0.6)	78.4 (−0.1)
Test	sqlparse	94.0 (+7.1)	75.4 (+1.6)	83.7 (+3.9)	94.8 (+7.1)	74.7 (+1.5)	83.5 (+3.7)	95.5 (+0.3)	51.2 (−2.9)	66.7 (−2.3)
Test	sshtunnel	96.7 (+14.4)	88.4 (+1.8)	92.3 (+7.9)	90.8 (+12.5)	84.3 (+0.6)	87.4 (+6.5)	100.0 (–)	61.6 (−9.3)	76.3 (−6.7)
Test	textrank4zh	92.9 (+4.7)	86.8 (+0.6)	89.8 (+2.6)	92.9 (+4.7)	86.8 (+0.6)	89.8 (+2.6)	100.0 (–)	73.6 (−3.4)	84.8 (−2.2)
Train Avg	–	93.2 (+4.4)	82.3 (+1.1)	87.0 (+2.3)	92.1 (+3.7)	77.6 (+0.6)	83.8 (+2.0)	97.6 (–)	61.7 (−6.3)	76.4 (−3.3)
Test Avg	–	94.2 (+6.8)	82.6 (+1.1)	88.0 (+3.8)	93.1 (+6.2)	80.4 (+0.6)	86.2 (+3.2)	98.2 (+0.4)	62.9 (−4.5)	76.5 (−3.0)
Overall Avg	–	94.0 (+5.8)	82.4 (+1.1)	87.8 (+3.3)	93.0 (+5.3)	78.9 (+0.6)	85.2 (+2.7)	97.9 (+0.1)	64.1 (−3.3)	77.1 (−2.4)

In terms of precision, InferCG generally lags behind PyCG and Depends, both of which adopt conservative static analysis strategies that naturally yield higher precision. However, our investigation indicates that part of this precision gap may stem from incompleteness in the macro-benchmark's ground-truth annotations. To examine this hypothesis, we systematically reviewed the false-positive edges produced by InferCG. Specifically,

Just Accepted

we applied a systematic multi-reviewer validation protocol. First, we aggregated all candidate call edges produced by the evaluated tools and exhaustively inspected them against the corresponding source files to determine whether each edge represented a semantically valid call. Two reviewers independently examined every edge, and any disagreements or ambiguous cases were forwarded to a third reviewer for adjudication. We did not compute formal inter-rater agreement metrics, as the number of disputed cases was small; nevertheless, the limited volume of escalations suggests a high degree of consistency across reviewers. Importantly, this systematic review revealed that many edges originally labeled as false positives were in fact legitimate call relationships that had been omitted from the benchmark annotations. Table 2 reports the updated results after incorporating these missing but valid edges into the ground-truth set. Values in parentheses represent absolute changes relative to the uncorrected benchmark. Because a substantial fraction of these previously unannotated call edges were detected by InferCG but not by PyCG, InferCG's precision and recall both rise noticeably, whereas PyCG's precision remains largely stable and its recall often decreases. For example, with DeepSeek as the pruning model, InferCG reaches a recall of 82.6%—an improvement of 19.7 points over PyCG—and an F1 score of 88.0%, outperforming PyCG by 11.5 points. Although Salis et al. [53] reported approximately 10 hours of manual inspection per project and followed a careful annotation methodology, our findings suggest that the macro-benchmark still exhibits non-negligible annotation gaps. This is consistent with the inherent difficulty of exhaustively enumerating call edges in large and highly dynamic Python systems.

To make the nature of the missing call edges more concrete, we provide an illustrative example from the `fabric` project identified during our manual review. In this case, the project interacts with the external `paramiko` library through an SSH client abstraction. As shown below, the `Connection` class initializes a transport object by invoking `SSHClient.get_transport()`, whose return value is an instance of `paramiko.transport.Transport`:

```
from paramiko.config import SSHConfig

class Connection(Context):
    client = SSHClient()
    self.client = client
    self.transport = self.client.get_transport()
    def create_session(self):
        channel = self.transport.open_session()
        if self.forward_agent:
            self._agent_handler = AgentRequestHandler(channel)
        return channel
```

This code induces a valid call edge from `Connection.create_session` to `paramiko.transport.Transport.open_session`. However, this edge was absent from the original macro-benchmark annotations. After correction, this call edge is included in the updated ground truth. InferCG successfully recovers this edge by combining coarse-grained candidate generation with LLM-based semantic reasoning over return types and object usage, whereas purely static approaches fail to establish this linkage.

4.2.2 Threshold-Agnostic Performance via AUC. To complement the point estimates at the 75% confidence threshold and to holistically evaluate the quality of InferCG's confidence scoring across all possible thresholds, we compute the Area Under the Receiver Operating Characteristic (ROC) Curve (AUC) on the test set. The ROC curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) as the confidence threshold varies. A model that randomly classifies call edges would produce a diagonal line with an AUC of 0.5, while a perfect classifier would achieve an AUC of 1.0.

As shown in Figure 7, the ROC curve for InferCG instantiated with DeepSeek achieves an AUC of **0.7589** on the test set. This result, significantly above the random baseline (AUC = 0.5), demonstrates that the confidence

Just Accepted

scores output by the LLM-based pruner are meaningful and provide a reliable discriminant between true and false call edges. The superior AUC indicates that InferCG's performance advantage is not an artifact of a single, optimally-chosen threshold but is robust across a wide range of threshold values.

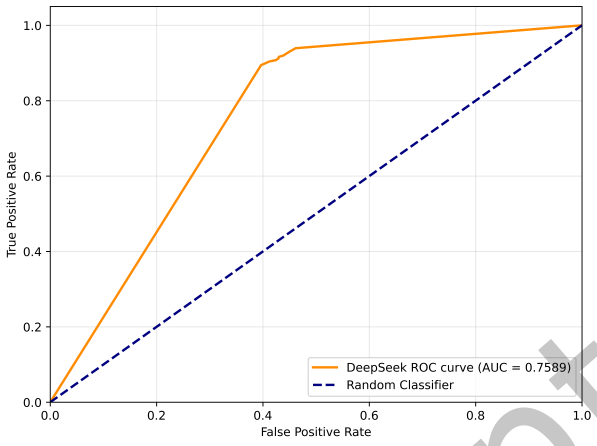


Fig. 7. DeepSeek: ROC Analysis Across Confidence Thresholds

4.2.3 DyPyBench Results. Building on DeepSeek's superior performance observed in the macro-benchmark, we employ DeepSeek as the LLM backend for InferCG in our DyPyBench evaluation. DyPyBench is a dynamic benchmark that collects ground-truth call graphs through runtime instrumentation and test-driven execution, which represent a complementary evaluation setting that emphasizes runtime-invoked call paths. Table 3 presents a comparative analysis of InferCG (using DeepSeek) and two classical static analyzers, PyCG and Depends, across four LLM confidence thresholds: 25%, 50%, 75%, and 100%.

Table 3. Results of two baseline tools and InferCG on the DyPyBench

Tool	Threshold	Precision	Recall	F1
PyCG	—	17.5	51.1	21.6
Depends	—	29.7	14.8	17.2
InferCG(DeepSeek)	25%	20.4	66.6	25.3
	50%	20.7	66.6	25.5
	75%	20.8	66.5	25.6
	100%	21.2	65.4	25.7

Overall, all three tools demonstrate relatively low precision on DyPyBench. This can be primarily attributed to the construction of the benchmark itself: since the ground-truth call graphs are derived through dynamic analysis, they are inherently constrained by the coverage of the test suite used to drive execution. Empirical data from the DyPyBench paper [8] indicate that the test suites achieve an average code coverage of 82% (i.e., 558k executed lines out of 681k total lines), meaning that 18% of the code remains unexecuted. Consequently, valid call edges in uncovered code paths are not recorded in the ground truth, leading to inflated false positive rates during

Just Accepted

evaluation. This limitation is further evidenced by the comparison of dynamic and static call graphs in DyPyBench, where dynamic analysis captured only 9,575 edges compared to 60,565 edges in static analysis, highlighting the substantial coverage gap. To provide additional empirical support for this explanation, we manually inspected a random sample of edges predicted by InferCG that were labeled as false positives under the DyPyBench ground truth. This inspection revealed that a non-trivial portion of these edges correspond to semantically valid call relationships located in code paths that were not exercised by the benchmark's test suites. In several cases, the callees reside in conditionally executed branches or auxiliary utility functions that are reachable under alternative inputs but remain unobserved during dynamic execution. These findings substantiate that the low measured precision is, at least in part, an artifact of incomplete dynamic coverage rather than incorrect predictions. In such settings, recall becomes a more faithful indicator of a tool's effectiveness in uncovering actual call relationships, as it reflects the ability to recover edges that are empirically observed in executions while remaining robust to missing annotations in unexecuted code regions.

With respect to recall, InferCG significantly outperforms both PyCG and Depends across all thresholds. At the 75% threshold—selected for its empirically optimal balance—InferCG achieves a recall of 66.5%, outperforming PyCG by 15.4 percentage points and Depends by 51.7 points. This marked advancement demonstrates InferCG's capability to infer call relations, outperforming traditional static analysis methods. Additionally, the recall remains consistently above 65% across all confidence thresholds, attesting to the robustness and stability of InferCG's prediction mechanism. The negligible variation in performance under different thresholds (as summarized in Table 3) can be attributed to the distribution of confidence scores output by the LLM-based pruner. Specifically, we observe that the confidence scores are highly polarized, with the majority of predictions clustering near 0% or 100%. This bimodal distribution indicates that the LLM exhibits high certainty in its judgments, resulting in minimal changes to the set of retained edges when the threshold is adjusted. Consequently, the recall metric remains stable, reflecting a strong correlation between the LLM's confidence and the actual correctness of its predictions.

One major factor contributing to PyCG's low recall is its inability to resolve calls involving third-party packages or compiled modules when source code is unavailable. InferCG mitigates this limitation by leveraging the function library, which enables the recovery of such call edges through approximate static signatures and LLM-supported inference.

In terms of F1 score, InferCG demonstrates superior performance as well. At the 75% confidence level, it attains an F1 of 25.6%, improving over PyCG by 4.0 points and Depends by 8.4 points. This indicates that while InferCG prioritizes recall, it does not do so at the expense of excessive precision loss. In fact, precision slightly improves as the confidence threshold increases, ranging from 20.4% at 25% threshold to 21.2% at full certainty (100%), showing a smooth tradeoff curve.

These results further corroborate the value of LLM-based semantic reasoning in supplementing static analysis, particularly in benchmarks driven by incomplete runtime observations. By aligning prediction confidence with model-inferred uncertainty, InferCG offers users a principled mechanism for tuning the precision-recall tradeoff according to downstream application needs, such as test coverage analysis or security triage.

ANSWER: InferCG strikes a strong balance between precision and recall. On the macro-benchmark, it improves recall and F1 over PyCG by 13.9% and 5.0%, respectively. On DyPyBench, recall increases by 15.4%, validating InferCG's effectiveness and generalizability in real-world Python projects.

4.3 Vulnerable-Function Detection (RQ2)

Modern dependency management tools, such as GitHub's Dependabot, primarily operate at the package level: they raise security alerts when a project declares a vulnerable library as a dependency. However, such tools lack

Just Accepted

the granularity to assess whether any of the known vulnerable functions are actually reachable or invoked in the dependent codebase. Consequently, these alerts often lead to over-approximation, potentially overwhelming developers with warnings that lack actionable context.

In contrast, InferCG supports fine-grained, method-level call graph analysis that enables precise vulnerability reachability assessment. By statically resolving function-level call relationships—including those involving indirect or dynamically bound invocations—InferCG can identify whether a vulnerable function is transitively reachable from within the client project. This capability allows developers to distinguish between declared-but-unused vulnerable packages and those that expose real attack surfaces, thus facilitating more informed mitigation decisions.

To evaluate this functionality, we curated a benchmark of 10 real-world vulnerability advisories from the GitHub Security Advisory database. To ensure the objectivity of our evaluation and mitigate potential selection bias, these advisories were randomly selected rather than being hand-picked for ease of analysis. These advisories span 10 distinct vulnerable functions across 8 popular Python packages (*django*, *picklescan*, *InvokeAI*, *onnx*, *gradio*, *mindsdb*, *sqlalchemy*, and *tarfile*). For the client projects, we identified downstream repositories that explicitly declare a dependency on the vulnerable versions. Notably, we did not impose constraints on project popularity. By including projects of varying maturity and code quality, we aim to evaluate InferCG's robustness across the heterogeneous landscape of the open-source ecosystem. This resulted in a total of 116 downstream projects. Table 4 summarizes the distribution of vulnerabilities and dependents. To verify the correctness of our results, two authors independently performed manual inspections of the source code to confirm the reachability of the vulnerable functions.

Risky Functions (RF) are defined as functions within a dependent project that directly or transitively invoke a known vulnerable function. We further define **Impacted Dependents (IP)** as the number of downstream projects in which at least one such risky function is identified. These metrics serve to evaluate both the granularity and the propagation capabilities of a given call graph construction technique in the context of security auditing.

Table 4. The details of collected vulnerable dataset.

Project	Functions	Dependents	CVE(s)
django	2	18	CVE-2024-53907, CVE-2012-4520
picklescan	1	11	CVE-2025-1944
InvokeAI	1	11	CVE-2024-12029
onnx	2	14	CVE-2022-25882, CVE-2024-5187
gradio	1	14	CVE-2024-47870
mindsdb	1	15	CVE-2024-45853
sqlalchemy	1	17	CVE-2019-7164
tarfile	1	16	CVE-2007-4559
Total	**10**	**116**	–

Table 5 presents the results of vulnerability reachability analysis conducted using Depends, PyCG, and InferCG. For each tool, we report the number of impacted dependents, risky functions, true positives (TP), and false positives (FP). Overall, Depends identifies only a small number of vulnerable propagation cases, reflecting its limited ability to capture inter-procedural and cross-package call relations in dynamic Python code. PyCG improves coverage by leveraging more precise static analysis, detecting 23 risky function invocations across 21 dependent projects, with no observed false positives. InferCG achieves the highest coverage, correctly identifying 30 risky function invocations spanning 28 dependent projects, also without introducing false positives. This is

Just Accepted

Table 5. Results for vulnerability propagation using Depends, PyCG, and InferCG.

Project	Depends			PyCG			**InferCG**		
	IP/RF	TP	FP	IP/RF	TP	FP	IP/RF	TP	FP
django	1/2	2	0	3/4	4	0	4/5	5	0
picklescan	0/0	0	0	6/7	7	0	7/8	8	0
InvokeAI	2/2	2	0	2/2	2	0	2/2	2	0
onnx	1/1	1	0	9/9	9	0	10/10	10	0
gradio	0/0	0	0	0/0	0	0	1/1	1	0
mindsdb	0/0	0	0	1/1	1	0	1/1	1	0
sqlalchemy	0/0	0	0	0/0	0	0	1/1	1	0
tarfile	0/0	0	0	0/0	0	0	2/2	2	0
Total	4/5	5	0	21/23	23	0	28/30	30	0

enabled by a combination of approximate function signature matching and LLM-based contextual reasoning, allowing InferCG to infer valid invocation relationships that other tools overlook.

Notably, among the 116 dependent projects, only 28 (approximately 24%) actually invoke the vulnerable functions. This finding confirms the over-approximation problem inherent in existing dependency-level vulnerability alerts. By contrast, InferCG narrows the alert scope to the subset of dependents that are semantically affected, thus reducing alert fatigue and enabling targeted remediation.

From a practical security engineering perspective, the ability to accurately assess vulnerability reachability is critical for prioritizing patching efforts. Tools like InferCG that integrate semantic reasoning and static analysis offer a promising direction toward actionable, high-precision vulnerability analysis pipelines.

ANSWER: InferCG accurately identifies vulnerable functions that are truly invoked, outperforming baseline tools with higher precision and no false positives. Its fine-grained, method-level analysis reduces over-approximation and significantly improves the accuracy of vulnerability reachability assessments. By uncovering actual call relationships, InferCG delivers more precise and reliable security insights than traditional static analysis tools.

4.4 Ablation Study (RQ3)

To better understand the contribution of key components in InferCG, we conduct a comprehensive ablation study focused on two critical design aspects: 1) the use of LLM-based pruning to refine the static call graph, and 2) the incorporation of global context information into the prompt construction for LLM inference.

Specifically, we evaluate the following configurations:

- **wo-Matching**: This configuration disables the function matching phase entirely, relying solely on call-path analysis to generate the call graph.
- **wo-LLM**: This configuration disables the LLM pruning phase entirely, relying solely on coarse-grained static analysis to generate the call graph.
- **wo-global**: This configuration retains LLM pruning but excludes global context (e.g., class definitions, enclosing methods) from the prompt provided to the language model.

Just Accepted

We perform experiments using two backbone LLMs: DeepSeek and Qwen. For Qwen, we set the temperature to 0 to ensure deterministic outputs. DeepSeek does not expose a temperature control parameter, so we use its default deterministic mode. In all settings, the LLM confidence threshold is fixed at 75%, as empirical results from Section 4.2 show that this value offers the best trade-off between precision and recall in call graph generation.

Table 6. Ablation Study on the Test Set

Method	Group	Precision	Recall	F1
PyCG	origin	97.9	66.7	79.2
	with-LLM	99.6	63.4	77.3
Coarse-Grained Static Analysis	wo-Matching	97.2	16.9	28.8
	wo-LLM	61.9	86.5	71.3
Random Pruning	origin	63.2	51.7	56.2
CodeBERT	origin	66.0	77.0	69.9
DeepSeek	origin	87.4	81.5	84.2
	wo-global	86.9	79.4	82.9
Qwen	origin	86.9	79.8	83.0
	wo-global	86.3	77.7	81.6

We first analyze the contribution of individual system components, as summarized in Table 6. When LLM pruning is turned off (`wo-LLM`), the system reverts to traditional coarse-grained static analysis. In this setting, recall remains high (86.5%), indicating that the tool captures a broad set of potential call edges. However, this comes at the cost of substantially reduced precision (61.9%), due to the inclusion of spurious or infeasible edges that static heuristics alone cannot eliminate. This result confirms the necessity of LLM-based pruning for improving the precision of the constructed call graph. In contrast, disabling the function matching phase (`wo-Matching`) leads to the opposite trade-off. Under this configuration, the analysis relies solely on intra-procedural call-path analysis and does not propagate call relationships across function boundaries. As a result, the generated call graph contains only call edges that are explicitly observable within individual functions, yielding very high precision (97.2%) but extremely low recall (16.9%). This behavior reflects the conservative nature of path-only analysis: while most inferred edges are correct, a large fraction of true inter-procedural call relations are missed due to the absence of cross-function matching. Taken together, these ablation results highlight the complementary roles of the two components.

To further disentangle the contribution of LLM-based pruning from that of the underlying static analysis, we additionally include a PyCG+LLM baseline, in which the same LLM pruning strategy (using DeepSeek) is applied directly to the call graph produced by PyCG. This comparison isolates whether the observed gains of InferCG arise from LLM-based refinement alone or from its specific integration with a recall-oriented coarse-grained analysis. As shown in Table 6, PyCG+LLM slightly improves precision over PyCG (99.6% vs. 97.9%) but further reduces recall (63.4% vs. 66.7%), leading to a lower F1 score (77.3%). This outcome is expected: PyCG already prioritizes precision, and additional LLM-based pruning primarily removes edges rather than recovering missing ones. These results suggest that LLM pruning alone cannot offset the recall loss introduced by conservative static analysis.

We further assessed the necessity of advanced LLM reasoning capabilities by comparing our approach against a weaker neural baseline (CodeBERT) and a randomized pruning strategy. CodeBERT, fine-tuned on the training

Just Accepted

portion of the macro-benchmark and DyPyBench, achieves an average F1 score of 69.9%, substantially below InferCG. This reflects the limited capacity of smaller pretrained models to perform deep semantic reasoning over long-range call contexts, resulting in imprecise filtering despite reasonable recall. Random pruning performs even worse (56.2% F1), confirming that naive edge reduction cannot account for the observed gains.

Once LLM pruning is enabled, both DeepSeek and Qwen exhibit substantial precision gains—with DeepSeek reaching 87.4% and Qwen achieving 86.9%—while maintaining recall levels above 79%. These results demonstrate that LLM pruning plays a crucial role in reducing false positives while retaining relevant call relationships.

The role of global context exhibits a more nuanced impact. For Qwen, removing global information from prompts (`wo-global`) leads to decreases in precision (-0.6%) and recall (-2.1%), resulting in a 1.4-point reduction in F1. Similarly, for DeepSeek, the absence of global information yields a modest decline in precision (-0.5%), recall (-2.1%), and F1 (-1.3 points). These results indicate that both models benefit from global contextual signals when resolving ambiguous or implicit call relationships, though the magnitude of the impact differs across models.

Upon manual inspection, we observe that global context is particularly useful for resolving implicit calls in object-oriented code, especially in cases involving constructors or inherited methods. Fig. 3 illustrates such an example: the presence of a `__init__` method in the global context enables InferCG to correctly identify that `ClassA.m1()` transitively invokes `func5()`, a link that static analysis alone fails to recover.

In summary, the ablation results underscore the complementary roles of global prompt information and LLM-based pruning. While the former enhances disambiguation and semantic understanding, the latter serves as a powerful filter to suppress noise introduced by static over-approximation. Together, they enable InferCG to produce high-quality, semantically grounded call graphs.

ANSWER: LLM pruning significantly improves precision by removing false positives from static analysis. Global context further enhances the precision and recall, especially for models like Qwen that rely more on contextual cues. DeepSeek, with stronger reasoning, is less affected by the absence of global information.

4.5 Exploring the Integration of InferCG and PyCG (RQ4)

Static and learning-based call graph generation techniques offer complementary strengths. On the one hand, PyCG leverages conservative static analysis to produce high-precision call edges, though at the cost of lower recall due to its limited handling of dynamic language features and incomplete code contexts. On the other hand, InferCG—powered by LLM-based semantic pruning—tends to achieve higher recall by exploiting contextual reasoning, but may also remove some legitimate call edges when the LLM misjudges ambiguous cases. Motivated by these complementary error profiles, we investigate whether combining the outputs of the two approaches can yield more comprehensive and accurate call graphs.

We adopt a simple yet effective *union-based integration strategy*, which constructs a merged call graph by aggregating all unique call edges identified by either PyCG or InferCG. Specifically, for each project under evaluation, we extract the call graphs from both tools and compute their edge-level union. During merging, the procedure iterates over the edge sets of both input graphs, ensuring that all involved source and target functions are preserved in the resulting node set. This strategy is recall-oriented: it admits any edge supported by at least one tool, thereby trading potential increases in false positives for broader functional coverage.

Table 7 presents the results of the merged graphs on the 11 macro-benchmark projects. As shown in Figure 6, the confidence threshold has only a marginal effect on the performance of InferCG. Consequently, tuning the threshold on the training subset is unlikely to materially influence the outcomes on the macro-benchmark. For this reason, we conduct the merging experiment directly on the full macro-benchmark rather than restricting the evaluation to the smaller training set. When integrating PyCG with InferCG (using either DeepSeek or Qwen), the combined tool consistently improves overall F1 performance across all projects. For example, the

Just Accepted

Table 7. Call graph quality of combining PyCG with InferCG (DeepSeek/Qwen) on the macro-benchmark (Precision / Recall / F1, in %).

Project	InferCG (DeepSeek) + PyCG			InferCG (Qwen) + PyCG		
	Precision	Recall	F1	Precision	Recall	F1
asciinema	90.6	88.1	89.3	91.0	89.6	90.3
autojump	93.0	86.9	89.8	92.7	85.7	89.1
fabric	77.5	84.0	80.7	77.9	82.2	80.0
face_classification	97.9	95.2	96.5	97.4	95.9	96.7
Sublist3r	95.0	88.6	91.7	95.3	88.3	91.7
bpytop	87.4	91.5	89.4	86.5	91.3	88.9
furl	90.7	76.5	83.0	92.6	77.2	84.2
rich_cli	90.0	95.5	92.7	91.0	92.4	91.7
sqlparse	85.6	79.9	82.7	86.1	79.7	82.8
sshtunnel	82.5	87.8	85.1	81.9	87.2	84.5
textrank4zh	91.3	98.8	88.2	92.3	98.8	95.5
Average	89.2	88.4	88.7	89.5	88.0	88.7

DeepSeek+PyCG combination achieves an average precision of 89.2%, recall of 88.4%, and an F1 score of 88.7%; similarly, Qwen+PyCG attains an F1 of 88.7%. Compared to using PyCG alone, this union strategy yields an average F1 improvement of 9.2 percentage points, confirming that the two approaches complement each other effectively.

Table 8. Results of PyCG combined with InferCG on the DyPyBench

Tool	**Threshold**	**Precision**	**Recall**	**F1**
InferCG+PyCG	25%	14.9	69.6	20.8
	50%	15.0	69.6	21.0
	75%	15.1	69.5	21.0
	100%	15.2	68.8	21.2

The trend is further validated on DyPyBench, as shown in Table 8. By merging InferCG (with DeepSeek as the LLM backend and threshold fixed at 75%) with PyCG, recall increases to 69.5%, representing an 18.4-point improvement over standalone PyCG. Although precision remains relatively low (15.1%), the substantial recall gain leads to a noticeable uplift in F1 score, confirming the utility of InferCG in augmenting the recall-limited nature of traditional static approaches.

We note that some limitations of InferCG—such as insufficient handling of Python-specific constructs like `lambda` expressions—restrict its precision and recall in isolation. Approximately 0.5% of the unresolved cases are attributed to `lambda` expressions, and an additional 6% result from incorrect removals of valid call edges by the LLM. However, these limitations are partially offset when combined with PyCG, which excels at resolving such syntactic constructs through AST traversal. Therefore, the merged call graph benefits from the strengths of

Just Accepted

both paradigms: InferCG contributes broader semantic coverage via LLM-based inference, while PyCG ensures syntactic precision and structural completeness. A more detailed discussion of InferCG's limitations and potential remedies is provided in Section 5.

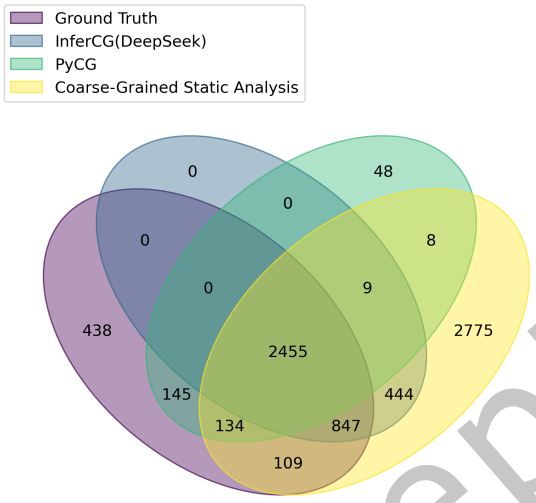


Fig. 8. Venn diagrams of edges in Ground Truth,InferCG(DeepSeek),PyCG,Coarse-Grained Static Analysis.

To further understand how different analysis techniques relate to one another, we provide a set-theoretic comparison of the call edges produced by four representative methods: the ground-truth annotations, InferCG (DeepSeek), PyCG, and the coarse-grained static analysis. The Venn diagram in Figure 8 visualizes their overlaps and divergences, revealing several noteworthy patterns. First, a substantial portion of edges recovered by InferCG coincide with the ground truth but are missed by PyCG, highlighting the benefits of LLM-driven semantic filtering in resolving dynamic or context-dependent call relations. Second, the coarse-grained static analysis covers a much larger superset of edges, many of which do not appear in any of the more precise methods; this confirms its role as a high-recall but low-precision foundation for subsequent pruning. At the same time, the diagram also shows nontrivial regions where PyCG identifies correct edges that InferCG does not retain, suggesting that the conservative nature of static analysis can be advantageous in cases where the LLM incorrectly prunes true call edges. The limited yet meaningful intersection between these two methods indicates complementary strengths: InferCG excels at recovering dynamic calls that evade static reasoning, whereas PyCG provides coverage for certain structurally evident edges that the LLM may discard under uncertainty. This complementary error profile motivates combining the two approaches and helps explain why their union yields higher overall F1 scores than either technique alone on several projects.

ANSWER: The integration of InferCG and PyCG demonstrates a promising hybrid strategy for call graph construction, effectively reconciling the trade-off between precision and recall by leveraging the complementary nature of static and LLM-based techniques.

Just Accepted

4.6 Computational Cost and Token Consumption

Beyond reporting accuracy, we additionally quantify the computational overhead associated with the LLM components of InferCG. On the macro-benchmark, InferCG consumes approximately 7043551 tokens in total across all projects. All LLM queries are issued through a discrete batch-inference pipeline (Batch API), which reduces monetary cost by roughly 50% compared to issuing individual requests. However, the Batch API returns results only after the batch-processing interval completes—typically around three hours—regardless of the number of tokens submitted or the complexity of the input. Because this fixed delay dominates wall-clock time, reporting execution latency would not meaningfully reflect InferCG's intrinsic computational requirements. Moreover, InferCG operates as an integrated hybrid system: coarse-grained static candidate generation and LLM-driven semantic filtering jointly form a single end-to-end workflow. Measuring the runtime of either component in isolation would therefore provide a misleading characterization of the overall system. For these reasons, we report token consumption—the principal variable cost in LLM-assisted analysis—and omit separate timing measurements.

5 DISCUSSION

5.1 Strengths of InferCG

InferCG introduces two key innovations that jointly contribute to improved recall and precision over traditional static call graph analyzers. First, to improve recall, it employs coarse-grained candidate generation based on a permissive matching strategy. By constructing a comprehensive function library and applying AST-based heuristics, InferCG is capable of identifying a broad set of potential caller–callee relationships. This heuristic strategy significantly increases coverage, particularly in scenarios where conventional static tools fail due to under-approximation. Second, to address the resulting over-approximation and enhance precision, InferCG applies an LLM as a semantic filter. Rather than relying on hand-crafted rules, this LLM-based component discriminates between plausible and spurious edges based on the semantic content of the code, improving graph quality in a learning-based and adaptable manner.

5.2 Limitations of InferCG

Despite its demonstrated effectiveness, InferCG operates under specific design constraints. One such constraint is the exclusion of anonymous functions, specifically `lambda` expressions, from the function library. This represents a deliberate design trade-off based on empirical evidence: our analysis of the benchmarks indicates that `lambda` expressions constitute approximately 0.5% of potential call sites. Given their low prevalence, we prioritized the efficiency and simplicity of the static analysis over the marginal recall gains from supporting these constructs. While this choice streamlines the implementation, it may result in minor recall loss in projects that rely heavily on functional programming idioms.

In addition, the coarse matching strategy, while recall-friendly, introduces false positives by relying primarily on function name and arity. It lacks scope- and hierarchy-sensitive disambiguation, which increases the likelihood of matching unrelated functions that happen to share names across modules or class hierarchies. For example, when a class `PassiveDNS` inherits from `EnumeratorBase`, and both classes define a method named `extract_domains`, the coarse-grained static analysis may conservatively include both `PassiveDNS.extract_domains` and `EnumeratorBase.extract_domains` as potential callees. More generally, when widely used method names appear across multiple classes or dependencies, the candidate set can grow substantially. This issue would be further amplified if InferCG were to incorporate the full transitive dependency closure during call graph construction. In large Python ecosystems, transitive dependencies often introduce hundreds of additional packages, many of which define similarly named utility methods. Including these indirect dependencies would dramatically expand the candidate call set, introducing a large number of spurious edges that are unlikely to be reachable in practice. Such over-expansion would not only degrade precision but also

Just Accepted

significantly increase the workload of the LLM-based pruning stage, leading to a sharp rise in token consumption and computational cost. For this reason, InferCG currently restricts its analysis to direct dependencies, prioritizing a practical balance between coverage, precision, and scalability.

Finally, InferCG's AST-based analysis does not fully capture Python-specific dynamic features, such as reflective calls via `getattr`, monkey patching, or runtime code generation. These constructs can obscure true call relationships and lead to incomplete or imprecise call graphs, particularly in dependency-heavy or highly dynamic codebases.

5.3 Summary and Future Directions

In summary, InferCG demonstrates that combining coarse-grained static analysis with LLM-guided semantic filtering can achieve a favorable balance between recall and precision for call-graph construction in Python. While our current implementation strategically omits low-frequency constructs like `lambda` expressions to optimize performance, limitations regarding the shallow modeling of dynamic behaviors and heuristic matching remain. Future work will explore finer-grained static analyses with control- and data-flow sensitivity, along with enhanced LLM-based pruning that incorporates inter-procedural context to extend coverage to anonymous functions and resolve complex dynamic dispatch scenarios.

Beyond these static-analysis–oriented improvements, it is also important to clarify the scope of InferCG. The current design targets pure Python programs and does not model interactions with native extensions or polyglot execution environments. However, real-world Python systems frequently rely on native libraries (e.g., C/C++ backends in numerical and machine-learning frameworks). Recent advances in cross-language analysis, such as CHARON [54], which uses polyglot property graphs for static vulnerability detection in native extensions, and PyXray [4], which leverages dynamic object layout analysis to efficiently construct Python-native call graphs, have demonstrated that such approaches can substantially improve coverage and accuracy. Incorporating such polyglot reasoning remains an open challenge for InferCG.

A second complementary direction concerns the integration of dynamic analysis. Systems such as DynaPyt [20] show that dynamic instrumentation can capture highly dynamic language constructs—including reflection, monkey patching, and data-dependent dispatch—that static analysis alone often under-approximates. Although dynamic techniques cannot observe unexecuted code paths, their precision on executed behaviors suggests that hybrid static–dynamic designs may offer a more complete solution. Integrating dynamic traces or polyglot analysis techniques with InferCG's static framework represents a promising path toward expanding its applicability and improving call-graph fidelity in complex, heterogeneous Python ecosystems.

6 THREATS TO VALIDITY

Internal Validity. In both our evaluation of vulnerable-function detection and the corrected macro-benchmark, the ground truth for call edges is established through manual annotation. Despite rigorous multi-round cross-validation among the authors, the process remains inherently subjective. Ambiguities in call-path boundaries—such as implicit function invocations or dynamic call contexts—can lead to inconsistent interpretations. These inconsistencies may introduce noise into the evaluation, thereby affecting the accuracy of reported precision, recall, and F1 scores. We acknowledge this limitation and emphasize that further refinement through community benchmarking or crowdsourced annotation would be valuable in future work.

External Validity. InferCG's semantic pruning depends on a pretrained LLM, whose generalization performance is influenced by its training corpus. While we selected two widely used open-source models (DeepSeek and Qwen), we cannot conclusively verify that their pretraining data excluded the benchmark projects used in our evaluation—namely, the 11 macro-benchmark repositories and DyPyBench samples. If any of these projects were seen during pretraining, the models may exhibit elevated inference accuracy on familiar code patterns, thus

Just Accepted

inflating the perceived effectiveness of InferCG. Although we designed our prompts and candidate generation process to mitigate direct memorization effects, this potential data overlap remains a threat to generalizability.

7 RELATED WORK

7.1 Points-to Analysis

Recent advances in pointer analysis address precision–scalability tradeoffs through three key directions. Sui et al. [55] propose demand-driven approaches that achieve IDE-friendly efficiency via sparse value-flow refinement, answering queries with 97.4% whole-program precision while limiting memory to 65KB per query. Context sensitivity innovations evolved from early partial transfer functions by Wilson et al. [62] to hybrid strategies: Kastrinis et al. [30] combine call-site and object sensitivity with selective context switching, achieving 3.05× speedup while maintaining precision. Precision-guided methods by Li et al. [39] optimize this further, applying context sensitivity to only 38% of methods via flow pattern recognition in precision flow graphs, preserving 98.8% precision with 3.4× faster analysis. Scalability breakthroughs introduced by Dholakia et al. [19] tackle million-line C programs through lazy/hybrid cycle detection, outperforming prior work by 20.6× while exploring memory-efficient BDD representations. These works collectively demonstrate the field's progression from whole-program analysis to targeted precision mechanisms and constraint resolution optimizations, enabling practical pointer analysis for modern software scales.

7.2 Type Inference for Dynamic programming languages

Type inference for dynamic programming languages has emerged as an important research area, with approaches ranging from static analysis to machine learning techniques. Salib et al. [52] introduced Starkiller, a static type inferencer using an enhanced Cartesian Product Algorithm to handle Python's dynamic features like dynamic class membership. While effective for constrained cases, purely static approaches struggle with language dynamism and external API dependencies. Xu et al. [63] addressed these limitations through probabilistic methods by modeling type uncertainty and proposing a graphical model aggregating type hints from data flow, attributes, and naming conventions. This approach showed superior coverage over static analyzers but suffered from combinatorial complexity. Peng et al. [49] introduced a recent hybrid approach, HiTyper, which combines static and probabilistic reasoning using Type Dependency Graphs to integrate static constraints with neural predictions, achieving a 30% improvement on rare types through iterative verification. Hellendoorn et al. [25] brought significant advances in type inference accuracy by developing DeepTyper using bidirectional RNNs with consistency layers for JavaScript/TypeScript, demonstrating 95% precision on compiler-verified types. For Python, Cui et al. [15] employed GRU networks with Byte-Pair Encoding to handle rare types, while Mir et al. [46] achieved state-of-the-art 77.1% MRR through similarity learning and type-checked datasets. These learned models capture contextual patterns but require validation against type systems to ensure correctness.

The evolution of type inference techniques reflects an increasing synthesis of static and learned approaches. Peng et al. [49] and Mir et al. [46] demonstrate how combining symbolic reasoning with neural predictions can overcome individual method limitations, particularly for handling dynamic language features and rare user-defined types.

7.3 Python Dependency Analysis and Dynamic Analysis Frameworks

Several empirical studies have investigated systemic risks and dependency-related vulnerabilities in Python packages. Mahon et al. [44] characterize structural weaknesses in Python's dependency management, showing how dependency confusion, implicit version resolution, and ecosystem-level practices can lead to widespread supply-chain risks. Their findings underscore that dependency analysis in Python must account not only for syntactic import relations but also for ecosystem-driven behaviors and deployment configurations. Complementary to

Just Accepted

this perspective, Quan et al. [51] conduct a large-scale empirical study on vulnerabilities in Python packages, revealing that many security issues are tightly coupled with dependency structures and transitive dependencies, thereby motivating more accurate and semantics-aware dependency analysis techniques. Beyond empirical characterization, recent work has explored dynamic and hybrid analysis techniques to address the limitations of purely static dependency analysis. Zheng et al. [66] propose a dynamic analysis pipeline for detecting supply-chain poisoning attacks in open-source ecosystems, including PyPI. Their approach emphasizes behavior-driven analysis under realistic execution environments, demonstrating that dynamic execution can expose malicious behaviors that remain hidden under static inspection alone. While their primary focus is on security detection rather than dependency modeling, the work illustrates the growing importance of execution-aware analysis in understanding dependency-related risks.

At the analysis infrastructure level, general-purpose dynamic analysis frameworks further enable fine-grained reasoning about Python program behavior. DynaPyt [20] provides a flexible framework for instrumenting Python programs and observing runtime events, facilitating analyses such as dynamic call graph construction and dependency tracking. More recent efforts combine static and dynamic views to reconstruct higher-level architectural and dependency information from Python systems [33], suggesting that hybrid approaches can mitigate the inherent imprecision of static-only analyses in dynamic languages.

7.4 Deep Learning-Enhanced Call Graph Analysis

Traditional static call graph construction faces fundamental precision–soundness trade-offs, with context-insensitive analyses like those in WALA [22] and Doop [9] achieving scalability at the cost of precision (70–76% false positive rates [57]). While context-sensitive analyses and reflection handling techniques provide incremental improvements, recent machine learning approaches have demonstrated transformative potential. Utture et al. [57] pioneered call graph pruning using Random Forests with 11 structural features (node degrees, call-site fanout, etc.), reducing false positives by 50% while maintaining 66% recall. However, their structural-only approach struggles with semantically similar true/false edges, particularly in polymorphic call scenarios [34]. Building on this foundation, Le-Cong et al. [34] introduced semantic-aware pruning through CodeBERT [21] embeddings, achieving a 13% F-measure improvement by combining code semantics with structural features. More recently, Mir et al. [45] conducted a large-scale empirical study on the effectiveness of machine learning-based call graph pruning, systematically reassessing prior conclusions on more realistic benchmarks. By introducing the NYXCorpus dataset composed of real-world Java projects with high test coverage, their study highlights that ML-based pruning yields substantial precision gains at the cost of moderate recall loss, and shows that carefully tuned conservative pruning strategies can produce call graphs comparable in quality to context-sensitive 1-CFA analyses while being significantly more efficient. Their findings further demonstrate that pruned call graphs can enable faster downstream security analyses with minimal loss in soundness, thereby reinforcing the practical viability of ML-based pruning in realistic analysis pipelines. This hybrid line of work addresses key limitations of graph neural networks [31] that tend to overfit structural patterns. Client analysis evaluations demonstrate concrete benefits: while CGPruner [57] reduces null pointer analysis false alarms from 73% to 23%, AutoPruner [34] achieves 12% through semantic-aware pruning, nearing the 15–20% threshold developers find acceptable [14]. The evolution from structural [57] to semantic-aware [34] pruning mirrors broader trends in ML-for-code research, where transformer models [58] now complement traditional program analysis [23]. Unlike hybrid static–dynamic approaches [6] requiring continuous instrumentation, both CGPruner and AutoPruner use dynamic analysis only during training, achieving production-ready static analysis—AutoPruner's key innovation being its code-aware pruning without runtime overhead.

Just Accepted

7.5 LLM-assisted Program Analysis

Recent advances demonstrate how LLMs overcome limitations of traditional static analysis by leveraging semantic understanding capabilities trained on massive code corpora. In vulnerability detection, *Wang et al.* [60] pioneered resource leak detection through InferROI's LLM-based intention inference, achieving 59.3%-62.5% bug detection rates when combined with path validation, while *Sun et al.* [56] decomposed smart contract vulnerabilities into code-level scenarios matched by GPT and confirmed via static data-flow checks in GPTScan, detecting 9 novel bugs with 65.8% false positive reduction. For fuzzing enhancement, *Yang et al.* [64] automated syscall specification synthesis in KernelGPT through iterative LLM analysis of kernel source code and validation-repair loops, achieving achieve higher coverage than state-of-the-art techniques, whereas *Deng et al.* [18] leveraged generative (Codex) and infilling (INCODER) models in TitanFuzz to create diverse DL API sequences that improved coverage by 50.84% and detected 44 new bugs. In code refactoring, *Pomian et al.* [50] combined LLM suggestions with IDE static analysis in EM-Assist, pruning 76.3% hallucinations via program slicing to achieve 53.4% accuracy in replicating developer extractions—surpassing static analyzers by 14%. For code completion, *Li et al.* [38] integrated IDE-derived contexts (e.g., cross-file types) through chain-of-thought prompting in IDECoder, improving syntax match accuracy by 6.56% over retrieval-augmented baselines. Collectively, these works establish that LLMs excel at semantic inference (intention recognition [60], vulnerability decomposition [56]), while static analysis provides precision through path validation [64], dependency checking [50], and context filtering [38], revealing future opportunities for dynamic feedback loops and domain-specific tuning to strengthen this synergy.

8 CONCLUSION

We have presented InferCG, a hybrid framework for Python call-graph generation that combines coarse-grained static analysis with LLM–based pruning. Initially, InferCG constructs a high-recall candidate set of call edges via permissive AST-driven matching, then refines this set by invoking an LLM to semantically validate and prune edges, achieving a balanced trade-off between precision and recall. Compared to state-of-the-art static tools such as PyCG and Depends, InferCG consistently outperforms them on multiple open-source projects and standard benchmarks, delivering up to a 5.0% increase in average F1 score. Furthermore, in a real-world vulnerable-function detection task, InferCG demonstrates superior precision and actual reachability analysis, addressing the coarse-grained limitations of existing dependency-management alerts. Overall, InferCG introduces a new paradigm for static analysis augmented by LLM inference, opening promising directions for research and practical applications in static code analysis, software security, and program comprehension.

References

[1] 2025. Depends. https://github.com/multilang-depends/depends.
[2] 2025. Pyan. https://github.com/Technologicat/pyan.
[3] Hesham Abandah and Izzat M Alsmadi. 2013. Call graph based metrics to evaluate software design quality. (2013).
[4] Georgios Alexopoulos, Thodoris Sotiropoulos, Georgios Gousios, Zhendong Su, and Dimitris Mitropoulos. [n. d.]. PyXray: Practical Cross-Language Call Graph Construction through Object Layout Analysis. ([n. d.]).
[5] Gábor Antal, Péter Hegedűs, Zoltán Herczeg, Gábor Lóki, and Rudolf Ferenc. 2023. Is JavaScript Call Graph Extraction Solved Yet? A Comparative Study of Static and Dynamic Tools. *IEEE Access* 11 (2023), 25266–25284.
[6] Shay Artzi, Adam Kiezun, David Glasser, and Michael D Ernst. 2007. Combined static and dynamic mutability analysis. In *Proceedings of the 22nd IEEE/ACM International Conference on Automated Software Engineering*. 104–113.
[7] ENNA BASIC and ALBERTO GIARETTA. 2025. From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security. (2025).
[8] Islem Bouzenia, Bajaj Piyush Krishan, and Michael Pradel. 2024. DyPyBench: A benchmark of executable python software. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 338–358.
[9] Martin Bravenboer and Yannis Smaragdakis. 2009. Strictly declarative specification of sophisticated points-to analyses. In *Proceedings of the 24th ACM SIGPLAN conference on Object oriented programming systems languages and applications*. 243–262.

Just Accepted

[10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[11] Yulu Cao, Lin Chen, Zhifei Chen, Jiacheng Zhong, Xiaowei Zhang, and Linzhang Wang. 2024. Efficient Construction of Practical Python Call Graphs with Entity Knowledge Base. *International Journal of Software Engineering and Knowledge Engineering* 34, 07 (2024), 999–1024.

[12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[13] Baijun Cheng, Cen Zhang, Kailong Wang, Ling Shi, Yang Liu, Haoyu Wang, Yao Guo, Ding Li, and Xiangqun Chen. 2024. Semantic-enhanced indirect call analysis with large language models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 430–442.

[14] Maria Christakis and Christian Bird. 2016. What developers want and need from program analysis: an empirical study. In *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*. 332–343.

[15] Siwei Cui, Gang Zhao, Zeyu Dai, Luochao Wang, Ruihong Huang, and Jeff Huang. 2021. Pyinfer: Deep learning semantic type inference for python variables. *arXiv preprint arXiv:2106.14316* (2021).

[16] Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. 2023. Masterkey: Automated jailbreak across multiple large language model chatbots. *arXiv preprint arXiv:2307.08715*.

[17] Gelei Deng, Yi Liu, Kailong Wang, Yuekang Li, Tianwei Zhang, and Yang Liu. 2024. Pandora: Jailbreak gpts by retrieval augmented generation poisoning. *arXiv preprint arXiv:2402.08416* (2024).

[18] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*. 423–435.

[19] Utpal Dholakia, Leona Tam, Sunyee Yoon, and Nancy Wong. 2016. The ant and the grasshopper: understanding personal saving orientation of consumers. *Journal of Consumer Research* 43, 1 (2016), 134–155.

[20] Aryaz Eghbali and Michael Pradel. 2022. DynaPyt: a dynamic analysis framework for Python. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 760–771.

[21] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).

[22] Stephen Fink and Julian Dolby. 2012. WALA–The TJ Watson Libraries for Analysis.

[23] Neville Grech, George Fourtounis, Adrian Francalanza, and Yannis Smaragdakis. 2018. Shooting from the heap: Ultra-scalable static analysis with heap snapshots. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 198–208.

[24] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

[25] Vincent J Hellendoorn, Christian Bird, Earl T Barr, and Miltiadis Allamanis. 2018. Deep learning type inference. In *Proceedings of the 2018 26th acm joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 152–162.

[26] Dominik Helm, Sven Keidel, Anemone Kampkötter, Johannes Düsing, Tobias Roth, Ben Hermann, and Mira Mezini. 2024. Total Recall,? How, Good, Are, Static, Call, Graphs, Really,?. In *Proceedings of the 33rd ACM, SIGSOFT, International, Symposium, on Software, Testing, and Analysis,*. ACM, 112–123.

[27] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.

[28] Kaifeng Huang, Yixuan Yan, Bihuan Chen, Zixin Tao, and Xin Peng. 2023. Scalable and Precise Application-Centered Call Graph Construction for Python. *arXiv preprint arXiv:2305.05949* (2023).

[29] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

[30] George Kastrinis and Yannis Smaragdakis. 2013. Hybrid context-sensitivity for points-to analysis. *ACM SIGPLAN Notices* 48, 6 (2013), 423–434.

[31] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[32] Sriteja Kummita, Goran Piskachev, Johannes Spä, th, and Eric Bodden. 2021. Qualitative and quantitative analysis of callgraph algorithms for Python. In *2021 International conference on code quality (ICCQ)*. IEEE, 1–15.

[33] Daphné Larrivain, Shinhyung Yang, and Wilhelm Hasselbring. 2025. Dynamic and Static Analysis of Python Software with Kieker Including Reconstructed Architectures. *arXiv preprint arXiv:2507.23425* (2025).

Just Accepted

[34] Thanh Le-Cong, Hong Jin Kang, Truong Giang Nguyen, Stefanus Agus Haryono, David Lo, Xuan-Bach D Le, and Quyet Thang Huynh. 2022. Autopruner: transformer-based call graph pruning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 520–532.
[35] Haodong Li, Gelei Deng, Yi Liu, Kailong Wang, Yuekang Li, Tianwei Zhang, Yang Liu, Guoai Xu, Guosheng Xu, and Haoyu Wang. 2024. Digger: Detecting copyright content mis-usage in large language model training. *arXiv preprint arXiv:2401.00676* (2024).
[36] Yuxi Li, Yi Liu, Gelei Deng, Ying Zhang, Wenjia Song, Ling Shi, Kailong Wang, Yuekang Li, Yang Liu, and Haoyu Wang. 2024. Glitch tokens in large language models: Categorization taxonomy and effective detection. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2075–2097.
[37] Yuxi Li, Yi Liu, Yuekang Li, Ling Shi, Gelei Deng, Shengquan Chen, and Kailong Wang. 2024. Lockpicking llms: A logit-based jailbreak using token-level manipulation. *arXiv preprint arXiv:2405.13068* (2024).
[38] Yichen Li, Yun Peng, Yintong Huo, and Michael R Lyu. 2024. Enhancing llm-based coding tools through native integration of ide-derived static context. In *Proceedings of the 1st International Workshop on Large Language Models for Code*. 70–74.
[39] Yue Li, Tian Tan, Anders Mø, ller, and Yannis Smaragdakis. 2018. Precision-guided context sensitivity for pointer analysis. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (2018), 1–29.
[40] X Liu, P Xu, J Wu, J Yuan, Y Yang, Y Zhou, F Liu, T Guan, H Wang, T Yu, et al. [n. d.]. Large language models and causal inference in collaboration: A comprehensive survey. arXiv. 2024. *arXiv preprint arXiv:2403.09606* ([n. d.]).
[41] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, et al. 2023. Prompt Injection attack against LLM-integrated Applications. *arXiv preprint arXiv:2306.05499* (2023).
[42] Zhen Liu, Ruoyu Wang, Nathalie Japkowicz, Heitor Murilo Gomes, Bitao Peng, and Wenbin Zhang. 2024. SeGDroid: An Android malware detection method based on sensitive function call graph learning. *Expert systems with applications* 235 (2024), 121125.
[43] Wei Ma, Shangqing Liu, Zhihao Lin, Wenhan Wang, Qiang Hu, Ye Liu, Cen Zhang, Liming Nie, Li Li, and Yang Liu. 2023. Lms: Understanding code syntax and semantics for code analysis. *arXiv preprint arXiv:2305.12138* (2023).
[44] Jacob Mahon, Chenxi Hou, and Zhihao Yao. 2025. PyPitfall: Dependency Chaos and Software Supply Chain Vulnerabilities in Python. *arXiv preprint arXiv:2507.18075* (2025).
[45] Amir M Mir, Mehdi Keshani, and Sebastian Proksch. 2024. On the effectiveness of machine learning-based call graph pruning: An empirical study. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 457–468.
[46] Amir M Mir, Evaldas Latoš„ kinas, Sebastian Proksch, and Georgios Gousios. 2022. Type4py: Practical deep similarity learning-based type inference for python. In *Proceedings of the 44th International Conference on Software Engineering*. 2241–2252.
[47] Gail C Murphy, David Notkin, William G Griswold, and Erica S Lan. 1998. An empirical study of static call graph extractors. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 7, 2 (1998), 158–191.
[48] Thu-Trang Nguyen, Thanh Trong Vu, Hieu Dinh Vo, and Son Nguyen. 2024. An empirical study on capability of large language models in understanding code semantics. *arXiv preprint arXiv:2407.03611* (2024).
[49] Yun Peng, Cuiyun Gao, Zongjie Li, Bowei Gao, David Lo, Qirun Zhang, and Michael Lyu. 2022. Static inference meets deep learning: a hybrid type inference approach for python. In *Proceedings of the 44th International Conference on Software Engineering*. 2019–2030.
[50] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Timofey Bryksin, and Danny Dig. 2024. Together we go further: LLMs and IDE static analysis for extract method refactoring. *arXiv preprint arXiv:2401.15298* (2024).
[51] Haowei Quan, Junjie Wang, Xinzhe Li, Terry Yue Zhuo, Xiao Chen, and Xiaoning Du. 2025. An Empirical Study of Vulnerabilities in Python Packages and Their Detection. *arXiv preprint arXiv:2509.04260* (2025).
[52] Michael Salib. 2004. *Starkiller: A static type inferencer and compiler for Python*. Ph. D. Dissertation. Massachusetts Institute of Technology.
[53] Vitalis Salis, Thodoris Sotiropoulos, Panos Louridas, Diomidis Spinellis, and Dimitris Mitropoulos. 2021. Pycg: Practical call graph generation in python. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1646–1657.
[54] Raoul Scholtes, Soheil Khodayari, Cristian-Alexandru Staicu, and Giancarlo Pellegrino. 2025. CHARON: Polyglot code analysis for detecting vulnerabilities in scripting languages native extensions. In *2025 IEEE 10th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 153–168.
[55] Yulei Sui and Jingling Xue. 2017. Demand-driven pointer analysis with strong updates via value-flow refinement. *arXiv preprint arXiv:1701.05650* (2017).
[56] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. 2024. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
[57] Akshay Utture, Shuyang Liu, Christian Gram Kalhauge, and Jens Palsberg. 2022. Striking a balance: pruning false-positives from static call graphs. In *Proceedings of the 44th International Conference on Software Engineering*. 2043–2055.
[58] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, ukasz Kaiser, Ł, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[59] Ashwin Prasad Shivarpatna Venkatesh, Samkutty Sabu, Amir M Mir, Sofia Reis, and Eric Bodden. 2024. The emergence of large language models in static analysis: A first look through micro-benchmarks. In *Proceedings of the 2024 IEEE/ACM First International Conference on*

Just Accepted

AI Foundation Models and Software Engineering. 35–39.

[60] Chong Wang, Jianan Liu, Xin Peng, Yang Liu, and Yiling Lou. 2025. Boosting static resource leak detection via llm-based resource-oriented intention inference. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 668–668.

[61] Guanyu Wang, Yuekang Li, Yi Liu, Gelei Deng, Tianlin Li, Guosheng Xu, Yang Liu, Haoyu Wang, and Kailong Wang. 2024. Metmap: Metamorphic testing for detecting false vector matching problems in LLM augmented generation. In *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering*. 12–23.

[62] Robert P Wilson and Monica S Lam. 1995. Efficient context-sensitive pointer analysis for C programs. *ACM Sigplan Notices* 30, 6 (1995), 1–12.

[63] Zhaogui Xu, Xiangyu Zhang, Lin Chen, Kexin Pei, and Baowen Xu. 2016. Python probabilistic type inference with natural language support. In *Proceedings of the 2016 24th ACM SIGSOFT international symposium on foundations of software engineering*. 607–618.

[64] Chenyuan Yang, Zijie Zhao, and Lingming Zhang. 2025. Kernelgpt: Enhanced kernel fuzzing via large language models. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 560–573.

[65] Li Yu. 2019. Empirical study of python call graph. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1274–1276.

[66] Xinyi Zheng, Chen Wei, Shenao Wang, Yanjie Zhao, Peiming Gao, Yuanchao Zhang, Kailong Wang, and Haoyu Wang. 2024. Towards robust detection of open source software supply chain poisoning attacks in industry environments. In *Proceedings of the 39th IEEE/ACM international conference on automated software engineering*. 1990–2001.

Just Accepted